

ARCANE-Mithril

算法规范及支撑文档¹

本算法由 ARCANE 团队设计完成，成员按姓名字母顺序排列如下：

- Jose Maria Bermudo Mera, 比利时 PQShield 公司
- Joppe W. Bos, 比利时恩智浦半导体 (NXP Semiconductors)
- Angshuman Karmakar, 印度坎普尔理工学院 (Indian Institute of Technology Kanpur)
- Joost Renes, 美国恩智浦半导体 (NXP Semiconductors)
- Sujoy Sinha Roy, 奥地利格拉茨工业大学 (Graz University of Technology)
- Michiel Van Beirendonck, 比利时贝尔福特实验室 (Belfort Labs)
- Frederik Vercauteren, 比利时鲁汶大学 (KU Leuven)
- 王鹏 (Peng Wang), 中国科学院大学密码学院
- 王跃武 (Yuewu Wang), 中国科学院大学密码学院
- 郑昉昱 (Fangyu Zheng), 中国科学院大学密码学院

联系人： 王鹏 (Peng Wang)

电子邮箱： p-wang@ucas.ac.cn

网站： <https://soc.ucas.ac.cn/index.php/zh/ARCANE>
<https://www.pq-arcane.org>

¹本文档仅供参考，最终内容请以英文版为准。

目录

1	ARCANE 算法套件	1
2	简介	4
2.1	基本方法概述	4
2.2	与现有算法的比较	5
3	预备知识	9
3.1	符号约定	9
3.2	格基密码学	10
3.2.1	误差学习	10
3.2.2	舍入学习	10
3.2.3	舍入学习的安全性	11
3.3	作为结构化模的环	12
3.4	根式环 LWR	14
4	算法描述	15
5	安全性声明与分析	19
5.1	理论安全性	19
5.1.1	Mithril.PKE	20
5.1.2	Mithril.KEM	21
5.1.3	RR-LWR 实例的安全性	21
5.1.4	具体攻击估计方法	23
5.2	实现安全性	24
5.2.1	解密失败攻击	24
5.2.2	侧信道攻击	25
6	性能评估	25
6.1	算术实现	26
6.2	性能分析	26
6.3	性能测试	27
A	SageMath 脚本	30
A.1	安全性估计	30
A.2	失败概率	32
B	检查清单	37
C	专业术语对照表	39

1 ARCANE 算法套件

后量子密码学 (post-quantum cryptography) 需要能够抵抗量子敌手的算法, 以便在密钥交换 (或密钥封装) 和数字签名等应用中取代 RSA、ECC 等传统方案。ARCANE 套件包含两种算法:

- ARCANE-Mithril : 一种抗量子密钥封装机制 (Key Encapsulation Mechanism, KEM);
- ARCANE-Octarine : 一种抗量子数字签名 (Digital Signature, DS) 方案。

本节重点介绍 Mithril 与 Octarine 共有的主要特性。

创新性 其安全性基于根式环舍入学习 (Radical Ring Learning-With-Rounding, RR-LWR) 问题 [BRVW26]。该问题是环上舍入学习 (Ring Learning-With-Rounding, Ring-LWR) [BPR12] 的一个特例, 可作为模上舍入学习 (Module Learning-With-Rounding, Module-LWR) 的替代方案。它既继承了 Module-LWR 参数选择灵活的优点, 又保留了 Ring-LWR 的大部分性能优势。KEM 和 DS 方案均在如下根式环上运行:

$$\mathbf{S}_{q,n,k} = (\mathbb{Z}_q[y]/(y^n + 1))[x]/(x^k - y - 2). \quad (1)$$

其中, q 和 n 均为 2 的幂, k 为任意正整数。它与环 $\mathbf{S}_{q,n,k} \cong \mathbb{Z}_q[x]/((x^k - 2)^n + 1)$ 之间存在一个直接的同构 [BRVW26, 第 4 节], 因此构成一个 Ring-LWR 实例。环 $\mathbf{S}_{q,n,k}$ 是第 $2n$ 个分圆域的 k 次根式扩张。因此, 在需要时可采用数论变换 (Number Theoretic Transform, NTT) 或其他多项式乘法等标准实现技术, 而根式扩张又使矩阵生成比模结构更为高效。

我们选择 2 的幂而非素数作为 q , 因此 Mithril 在设计上类似于 Saber [DKR⁺20, 附录 A.2], 但二者选择的理想格不同 (RR-LWR 而非 Module-LWR)。我们还使用了一种成熟签名范式, 即带中止的 Fiat-Shamir (Fiat-Shamir with Aborts) 范式 [Lyu09, Lyu12], 定义了适用于 RR-LWR 场景的数字签名方案 Octarine。Okada 等人此前提出了基于 LWR 的签名方案 MLWRSig [OTF⁺20]。我们采取了不同的方法: 将 $\mathbf{s}_2$ 定义为公钥产生的确定性误差 $\mathbf{s}_2 = \mathbf{A}\mathbf{s}_1 - \frac{q}{p} \lfloor \frac{p}{q} \mathbf{A}\mathbf{s}_1 \rfloor$, 而不是他们文献中 MLWRSig 中的 $\lfloor \frac{p}{q} \mathbf{A}\mathbf{s}_1 \rfloor - \frac{p}{q} \mathbf{A}\mathbf{s}_1$ 。此外, 我们选择在密钥生成阶段而非签名生成阶段计算 $\mathbf{s}_2$ 。总体而言, 我们设计更简明, 且整体构造较 MLWRSig 更接近 CRYSTALS-Dilithium [DKL⁺18]。

简洁性 我们算法的困难问题使用经过文献广泛研究、困难性已获充分估计的 Ring-LWR 问题 [BBD⁺26]。对于 KEM 和数字签名方案, 我们采用标准方法, 将格问题转化为适应性选择密文攻击下的不可区分性 (indistinguishability under adaptive chosen-ciphertext attack, IND-CCA2) [BDPR98] 和适应性选择消息攻击下的强不可伪造性 (strong unforgeability under chosen-message attack, SUF-CMA) [BN00] 等保守安全性质。

我们从多个方面简化现有设计。第一, 以 (根式) 环结构代替模结构; 第二, 通过采用 2 的幂作为 q , 尽量降低各种参数选择的复杂度。所有采样均采用模 2 的幂 α 的区间 $[-\alpha, \alpha - 1]$ 上的均匀分布, 并尽可能在各参数集中统一 α 的取值。生成 $[-\alpha, \alpha - 1]$ 内的随机元素时, 只需直接生成 $\log_2(2\alpha)$ 个随机比特。因此这一过程既不需要拒绝采样, 也不必丢弃任何 (伪) 随机生

成的比特。我们选择的参数确保失败概率始终可忽略。在我们的数字签名方案中，分解参数和 Fiat-Shamir 循环中止相关的参数均基于 2 的幂，相较于 Dilithium 等算法复杂运算而言，只需检查是否产生进位即可轻松完成边界检查。我们还减少了签名生成过程中的拒绝次数，从而降低签名时间的方差，使该方案在应用场景下更为稳定。例如，Octarine-128 完成一次签名平均需要 2.13 轮，标准差为 1.55；相比之下，Dilithium2 平均需要 4.25 轮，标准差为 3.72。

灵活性 我们继承了格基密码系统的若干优点。将 q 选为 13 比特或 24 比特的 2 的幂，可使模 q 的值容纳于大多数现代 CPU 字长之内，因而易于实现。我们作了适应性设计，以提高跨架构安全、灵活实现的潜力：

- 素数模 q 下的 LWR 可使用渐近高效的 NTT，有利于高速实现。然而，已有研究表明，2 的幂形式的 q 更适合掩码防护 (masking) [CJRR99, PR13]。掩码通常用于抵御侧信道攻击和故障攻击，在嵌入式系统和高保障实现中尤其重要。作为构造掩码实现的常用工具，算术到布尔转换（及其逆转换）[CGV14, BBE⁺18] 在模数为 2 的幂时更为简单；无需拒绝采样的采样例程同样如此。嵌入式系统本就受到资源限制，因此简化侧信道与故障防护对策有助于降低实现成本。
- 采用根式环可缩小公开矩阵，直接降低存储需求。例如，与 ML-DSA-87 的矩阵相比，矩阵尺寸缩小至约 1/7：若每个系数使用 32 比特，后者需要超过 57.3 kB；以紧凑形式存储时也需超过 41.2 kB。相比之下，我们的公开矩阵在每个系数使用 32 比特时约占 8.2 kB，紧凑存储时仅占 6.1 kB。这不仅降低了矩阵生成成本，也减少了在多次使用期间保存矩阵所需的内存（尤其是在签名生成时），从而为资源受限系统提供更优的时间-内存权衡。
- 我们在标准域中表示所有变量，避免算法描述依赖特定实现或算法技术。虽然以特定方式（例如 NTT 域）表示元素可能带来性能优势，但在面向不同平台时也可能限制算法的可移植性。ARCANE 套件的算术支持使用 NTT [CT65, Pol71]、Toom-Cook [Too63, Co066]、Karatsuba [KO62] 等不同算法高效实现，也可利用现有硬件执行 Kronecker 代换 [Kro82, BRvV22]。我们证明，其性能与当前先进的格基密码系统相比仍具竞争力。特别地，将 Mithril 私钥多项式 s 表示在标准域而非 NTT 域中后，每个系数只需 $\log_2(2\eta)$ 比特，而 Kyber 需要 $\log_2(q)$ 比特，因而 Mithril 算法可显著缩小私钥。

兼容性 Ring-LWR 的安全等级主要由格维数决定，而格维数取决于多项式次数 n 和根式扩张次数 k 。增大 n 可提高算术运算的整体效率，而改变 k 则可像 Module-LWR 一样对安全等级进行更细粒度的控制。KEM 通常选择 $n = 256$ ，但这会使达到 128 比特安全等级变得困难，因为此时格维数只能选择 512 或 768。维数 512 可能无法提供足够安全性（或安全裕量极小），而维数 768 通常又提供了过高的安全等级（从而影响性能）。因此，我们对 KEM 采用保守设计，选择 $n = 128$ 、 $k = 5$ ，在维数为 640 的格上工作。具体而言，在精化攻击模型下，我们算法的比特安全性能从类似算法 Kyber 和 Saber 的 140 比特提高到更安全的 153 比特。基于根式扩张的理想格可由一个随 k 呈线性增长的元素表示，而 Module-LWE/LWR 中则随 k 呈二次增长，因而缓解了对矩阵生成成本的部分影响。与使用较小格相比，我们的设计策略会略微增加密钥和密文尺寸并影响整体性能；然而这能显著增强我们对达到 128 比特安全等级的信心。

我们在所有 KEM 参数集中统一采用 $n = 128$ ，以简化软硬件实现。对于数字签名而言，我们则所有参数集中选择 $n = 1024$ ，以充分利用分圆子环结构提高性能，并令 k 分别取 1, 2, 5。我们的签名方案中有许多参数可供调节，它们会影响性能、密钥尺寸、拒绝率、签名时间方差等。我们提交方案中选择的参数使公钥与签名尺寸同其他格基提案相近，能同时提高性能，并降低部分常用参数集的签名生成方差。例如，将 ARCANE-Octarine-128 与 CRYSTALS-Dilithium-2 [DKL⁺18] 比较，签名生成的重复次数的期望能够减少约一半（分别为 2.1 次和 4.3 次）。

为最大限度复用数字签名与 KEM 的实现，我们将所有采样分布均选为模 2 的幂 α 的区间 $[-\alpha, \alpha - 1]$ 上的均匀分布。数字签名方案和 KEM 均默认选择 $\alpha = 2$ 的采样分布；仅在一个参数集中因安全影响过大而例外。基于 2 的幂的打包和舍入运算效率很高，并能最优利用所需比特。在两种方案中，我们均选择对 NTT 友好的次数 n （即 2 的幂），从而在 NTT 适合给定平台时采用基于 NTT 的算术。虽然 KEM 与数字签名中的 n 不同，但可将 $n = 128$ 的乘法作为次数 1024 乘法的子程序，实现共享。举例而言，KEM 取 $q = 2^{13}$ ，数字签名取 $q = 2^{24}$ 时，可选择对一个对 NTT 友好的 32 比特素数 p_1 来执行 NTT；该素数既可用于 KEM，也可利用环 $\mathbb{Z}_{p_1 p_2} \cong \mathbb{Z}_{p_1} \times \mathbb{Z}_{p_2}$ 用于数字签名。实现时可用单个模块在任意素数 p 的 $\mathbb{Z}_p$ 中运算，复用大部分代码；再在其上添加一个很小的双素数中国剩余定理（Chinese Remainder Theorem, CRT）计算层，将结果从 $\mathbb{Z}_{p_1} \times \mathbb{Z}_{p_2}$ 转换回 $\mathbb{Z}_p$ 。我们在参考实现和优化实现中展示了这种方法及其效率。

需要说明的是，素数的选择乃至是否使用 NTT 均不属于本算法实现规范的一部分，仅仅是一种实现上的选择。使用者可根据实际情况采用其他实现方法。

可扩展性 这些构造旨在成为可复用的基础组件，而限于本文档规定的具体实例。只要外围协议提供必要的认证、密钥确认及组合安全保证，KEM 就可集成至临时密钥建立、KEM-DEM 加密和认证密钥建立机制等高层协议中。它还可在混合模式下同经典或其他后量子密钥建立机制组合，以支持迁移期间的算法敏捷性。

方案的模块化组织也为未来变体留出了空间。特别地，可以在不改变底层根式环运算的前提下考虑其他参数配置、哈希/XOF/KDF 实例或实现配置。不过，这些变体仍应附有更新后的正确性界、具体安全性估计以及实现验证。

数字签名方案支持通用消息签名等标准应用，包括代码签名、安全证明和数字证书。此外，由于该方案遵循 Fiat-Shamir 范式，同一代数结构也可作为更高级 Fiat-Shamir 类构造的起点，包括门限签名 [PN25] 和认证加密 [AKSvV22]。

因此，RR-LWR 框架不应只被视为一组固定的 KEM 和签名实例，也应被视为支持未来协议集成及经过审慎分析的变体的平台；这些变体可以复用相同的根式环结构。

在不同平台上的性能优势 ARCANE 套件的保守设计选择使实现可针对广泛的目标平台进行优化。特别地，本规范有意将多项式算术方法的选择与算法本身解耦：实现可酌情采用基于 NTT、Toom-Cook、Karatsuba、教科书式乘法或特定架构的技术。

为同成熟的格基方案进行有意义的比较，我们用 SHAKE 可扩展输出函数（extendable-output function, XOF）[Nat15] 实例化 Mithril 和 Octarine。在 KEM 场景中，Mithril-128 的优化 AVX2 实现比 ML-KEM-512 慢约 1.4 倍，其中部分原因是 Mithril-128 的估计安全裕量大于 ML-KEM-512。在更高安全等级下，Mithril-256 与 ML-KEM-1024 的性能差距缩小至约 1.1 倍，同时 Mithril 仍提供更大的估计安全裕量。

在数字签名场景中，Octarine-128 的密钥生成和签名速度快于 ML-DSA-44，但 ML-DSA-44 的验证速度仍较快。在更高安全等级下，Octarine-256 在所测签名和验证操作上均优于 ML-DSA-87，签名和验证的加速比分别为 1.17 和 1.25。这些结果表明，ARCANE 设计在保留保守安全裕量和充分平台优化灵活性的同时，实现了有竞争力的性能。

我们注意到，在较小的 Arm 或 RISC-V 平台内核上，XOF 的成本占主导地位，因此在这些平台上 ARCANE 方案减少内存消耗、降低生成公钥矩阵所需伪随机性的优势将愈发明显。此外，简洁的设计选择应会显著有助于实现侧信道与故障防护；例如 Saber 相较于 Kyber 的优势已经表明了这一点。同时，本方案对硬件设计仍保持灵活，使用者可根据需求采用不同乘法器。我们计划在后续版本的规范中量化这些改进。

2 简介

本节将重点介绍密钥封装机制 ARCANE-Mithril（亦简称为 Mithril）。

2.1 基本方法概述

Mithril 的安全性基于根式环舍入学习（Radical Ring Learning With Rounding, RR-LWR）问题的困难性。直观的说，RR-LWR 是在根式环 $\mathbf{S}_{q,n,k}$ 上实例化的 LWR 问题，该环是对 2 的幂次分圆环的结构化扩张。给定从 $\mathbf{S}_{q,n,k}$ 导出的均匀随机公开数据，RR-LWR 要求将含秘密元素的舍入的线性乘积与均匀随机值区分开来，或恢复秘密本身。该假设引入一种介于分圆环与一般模格之间的根式环结构，从而推广了标准 Ring-LWR 与 Module-LWR 问题。该结构中，根式环 $\mathbf{S}_{q,n,k}$ 含有分圆子环 $\mathbf{R}_{q,n}$ ，因此既能支持高效算术，又保留了 LWR 构造所需的代数性质。从格的角度看，RR-LWR 可视为维数与 nk 成正比的格上的 LWR 实例，其中参数 k 为安全等级的调节提供了额外自由度。

Mithril 的构造未引入 RR-LWR 之外的新密码假设。密钥封装机制由公钥加密方案经 Fujisaki-Okamoto 型变换得到，其安全性依赖 RR-LWR 的假定困难性及所用哈希函数的标准性质。

环结构选择与参数选择准则 Mithril 旨在保留 LWR 构造简洁高效特点的同时实现灵活的参数选择。为此，方案在根式环 $\mathbf{S}_{q,n,k}$ 上实例化，其中 n 为基环次数， k 为根式扩张次数。

固定 2 的幂次基环次数 n 而改变 k ，会改变总代数次数 nk ，也即一般攻击估计采用的有效格维数。这样无需改变底层分圆次数即可细粒度调节安全性。子环 $\mathbf{R}_{q,n}$ 是 2 的幂次分圆环，支持高效多项式算术，并与成熟的 LWR 方案实现技术兼容。

模数 p 、 q 、 t 均选为 2 的幂，其比特长度分别记为 ϵ_p 、 ϵ_q 、 ϵ_t 。由此，LWR 构造中的舍入与缩放均可逐系数通过移位和截断实现。这些模数的相对大小既要保证以高概率正确解密，又要在噪声分量与消息分量之间留出充分间隔。

最后，参数 ℓ 决定加解密时从一个环元素中提取的系数数目。方案只处理关于 x 的次数最高 ℓ 个系数，在提供所需消息空间的同时避免无谓计算。 ℓ 的选择须与 n 、 k 及舍入参数相协调，以同时保证正确性和安全性。

参数集的完整概览见表 2。

失败概率 Mithril 底层公钥加密方案的正确性依赖其 LWR 构造的正确性。由于使用了舍入和系数截断，解密并非完全正确，而是以很小概率失败。

加密通过向舍入线性乘积加入缩放后的消息分量，将消息嵌入密文。解密通过逆转缩放和舍入操作恢复消息。当累积舍入误差使某个系数越过判决边界，从而错误解码相应比特时，即发生解密失败。

解密误差有两个来源：其一，LWR 舍入产生近似误差，大小取决于模数 p 与 q 的比值；其二，消息编解码只使用关于 x 的最高 ℓ 个次数系数，限制了可用误差裕量。参数 ϵ_p 、 ϵ_q 、 ϵ_t 和 ℓ 的选择会对此进行影响。

对于本文规定的参数集，解密失败概率大小受一个可忽略量约束，通常形如安全参数 λ 下的 $2^{-\lambda}$ 。密钥封装机制的设计已计入该概率，其中 Fujisaki–Okamoto 变换 [FO99] 保证解密失败不会损害所导出共享秘密的安全性。

因此，尽管加密方案并非完全正确，但其失败概率在预期安全模型中仍足够小，因此可以接受。在规定的 KEM 构造内使用时，非零失败概率不影响 Mithril 的整体安全性声明。

失败概率的详细分析见第 5.2.1 节。

采样 本方案广泛采用从二进制种子出发的确定性采样来生成公开矩阵和秘密值。该设计简化了密钥的存储与传输，实现了公开参数的紧凑表示，并保证不同实现的行为可复现。方案使用的全部随机性均通过明确定义的扩展函数从种子显式导出。

部分环元素 通过参数 ℓ 使用部分环元素可避免无谓计算。将消息编解码限制在关于 x 的最高 ℓ 个次数系数上，既降低了计算开销，又保留所需消息空间和正确性保证。与使用完整环元素相比，该选择不改变底层安全假设，便于高效实现，同时能降低失败概率。

Fujisaki–Okamoto 变换 公钥加密方案的结构兼容从 IND-CPA 安全加密到 IND-CCA 安全密钥封装机制的标准变换。特别地，加密算法可表示为显式使用随机性的形式，因而可在不修改方案核心算术的情况下应用 Fujisaki–Okamoto 型变换 [FO99]。

高保障实现 本算法规范支持常数时间实现，并简化针对侧信道和故障攻击的安全实现。所有算术运算均逐系数定义；密钥封装机制中的条件行为可用常数时间选择原语实现。这些设计有助于降低实际部署中的侧信道泄漏风险。

2.2 与现有算法的比较

下面将 Mithril 与最相关的现有格基 KEM 进行比较。本设计旨在更好地平衡概念简洁性、实现效率、内存占用以及加固实现的稳健性。我们的目标不仅是取得有竞争力的渐近安全性和性能，也希望降低方案在资源受限及侧信道敏感环境中的部署工程复杂度。与 LWR 类提案的详细比较见表 1。

CRYSTALS-Kyber / ML-KEM Mithril 与 ML-KEM（原名 CRYSTALS-Kyber）最显著的区别在于：Mithril 基于 Ring-LWR，而 ML-KEM 基于 Module-LWE。底层困难问题之间的关系详见第 3 节。

方案	$f(x)$	k	ℓ	ϵ_q	ϵ_p	ϵ_t	D_A	D_s	B
Mithril-128	$(x^k - 2)^{128} + 1$	5	1	13	11	3	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
Mithril-256	$(x^k - 2)^{128} + 1$	9	2	13	11	3	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
Mithril-512	$(x^k - 2)^{128} + 1$	17	4	13	11	8	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
LightSaber [DKRV18]	$x^{256} + 1$	2	1	13	10	3	$\mathcal{U}_{\epsilon_q}$	β_5	1
Saber [DKRV18]	$x^{256} + 1$	3	1	13	10	4	$\mathcal{U}_{\epsilon_q}$	β_4	1
FireSaber [DKRV18]	$x^{256} + 1$	4	1	13	10	6	$\mathcal{U}_{\epsilon_q}$	β_3	1
uLightSaber [DKR ⁺ 20, Sec. A.2]	$x^{256} + 1$	2	1	12	10	3	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
uSaber [DKR ⁺ 20, Sec. A.2]	$x^{256} + 1$	3	1	12	10	4	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
uFireSaber [DKR ⁺ 20, Sec. A.2]	$x^{256} + 1$	4	1	12	10	6	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_2$	1
Florete [MKKV21, KNK ⁺ 25]	$x^{768} - x^{384} + 1$	1	1	10	9	3	$\mathcal{U}_{\epsilon_q}$	β_4	1
Espada [MKKV21, KNK ⁺ 25]	$x^{64} + 1$	12	1	15	13	3	$\mathcal{U}_{\epsilon_q}$	β_4	4
LightSable [MKKV21, KNK ⁺ 25]	$x^{256} + 1$	2	1	11	9	2	$\mathcal{U}_{\epsilon_q}$	β_1	1
Sable [MKKV21, KNK ⁺ 25]	$x^{256} + 1$	3	1	11	9	4	$\mathcal{U}_{\epsilon_q}$	β_1	1
FireSable [MKKV21, KNK ⁺ 25]	$x^{256} + 1$	4	1	11	9	2	$\mathcal{U}_{\epsilon_q}$	β_1	1
TiMER* [HCCSS24]	$x^{256} + 1$	2	1	10	8	3	$\mathcal{U}_{\epsilon_q}$	spCBD	2
SMAUG-T128* [HCCSS24]	$x^{256} + 1$	2	1	10	8	5	$\mathcal{U}_{\epsilon_q}$	spCBD	1
SMAUG-T192* [HCCSS24]	$x^{256} + 1$	3	1	10	8	8	$\mathcal{U}_{\epsilon_q}$	spCBD	1
SMAUG-T256* [HCCSS24]	$x^{256} + 1$	5	1	10	8	6	$\mathcal{U}_{\epsilon_q}$	spCBD	1

表 1: 基于 Ring/Module-LWR 的后量子密码方案参数比较。除 Mithril 外, 所有方案均采用环 $\mathbf{R} = \mathbf{Z}[x]/(f(x))$ 上的 Module-LWR 参数、矩阵分布 D_A , 以及用于 $\mathbf{s}$ 和/或 $\mathbf{s}'$ 的 (临时) 私钥分布 D_s 。列 B 给出每个输入消息系数的比特数。 ϵ_q 、 ϵ_p 和 ϵ_t 分别表示 $\log_2 q$ 、 $\log_2 p$ 和 $\log_2 t$ 。对于 Module-LWR 方案, k 表示模秩; 对于 Mithril, 同一列表示根式扩张次数。

* 密钥生成采用 LWE, 封装/解封装采用 LWR。

与许多可将 q 选为素数的 LWE 类设计相比, LWR 类设计一个显著差异是: 由于 $\mathbf{Z}_q$ 中缺少适当的单位根, NTT 无法直接使用。这会影响 $\mathbf{R}_q$ 中多项式算术的效率, 使 LWR 实现通常慢于 LWE 实现。不过, 文献 [CHK⁺21] 首次表明, 可将 $\mathbf{R}_q$ 嵌入模数为素数 q' 的更大 NTT 友好环中, 从而仍可采用渐近更快的 NTT。虽然这会使模数比特长度至少加倍, 但在某些平台上仍可能优于 Toom-Cook 或 Karatsuba。以 Saber 和 FireSaber 参数集为例, 文献 [CHK⁺21] 使用 6 层长度为 256 的负循环 NTT, 随后执行 4×4 教科书式乘法, 所用模数为

$$q' = 25166081 = 196610 \cdot 128 + 1,$$

所得乘法比 $\mathbf{R}_{8192}$ 中的实现约快 60%。总体而言, LWR 与 LWE 的计算方式存在差异, 但性能足够接近, 甚至在许多实际应用中并不显著。

此外, 虽然 Mithril 基于 Ring-LWR 而非模问题, 但通过调节参数 k , 环 $\mathbf{S}_{q,n,k}$ 既继承 Module-LWR 参数选择灵活的优点, 又保留 Ring-LWR 的大部分性能优势。采用根式环可缩小公开矩阵, 直接降低采样开销和内存需求。所有采样均采用模二幂 α 的区间 $[-\alpha, \alpha - 1]$ 上的均

匀分布，并尽量在各参数集中统一 α 。生成该区间内的随机元素只需产生 $\log_2(2\alpha)$ 个随机比特，既不需要拒绝采样，也不必丢弃任何（伪）随机比特。

对于高保障实现，Mithril 等 LWR 类方案具有两项关键优势：

- 二幂模数可简化掩码转换；
- 舍入比噪声采样更易进行掩码防护。

这些性质促成了一阶掩码 Saber 框架：在 ARM Cortex-M4 上其开销为原始方案 2.5 倍，而 LWE 类方案为原始方案 5 倍。Saber 与 uSaber 的高阶掩码实现 [KDvB⁺22] 表明，随着阶数 t 增大，LWR 方案的扩展性优于 LWE 方案。均匀秘密分布因采样简单而最为高效。Scabbard 套件中基于 LWR 的 Florete、Espada 和 Sable 的掩码实现也因采用二幂模数而优于 ML-KEM；文献 [KKV23] 将其视为掩码友好方案。得益于高效参数选择，任意掩码阶数下，掩码 Florete 与掩码 Sable 甚至快于掩码 Saber。尽管文献 [KDvB⁺22] 将 uSaber 作为 Saber 的掩码友好版本，Sable 的性能仍优于 uSaber，且该优势在掩码后继续保持。Mithril 同样受益于这些优点。

在硬件方面，文献 [FVR⁺22] 针对 Kyber (ML-KEM) 和 Saber 的掩码实现提出 RISC-V 指令集扩展，结果证实 LWR 的掩码效率优于 LWE 方案。文献 [ABC⁺23] 进一步优化了二幂模数下的掩码多项式乘法；其 Saber 硬件仅以 1.7 倍面积开销实现一阶防护，并在 340 个周期内完成乘法，显著优于通用掩码方法。

最后，对于 128 比特参数集，我们采用安全性更保守的设计：选择 $n = 128$ 、 $k = 5$ ，在维数 640 的格上工作。在精化攻击模型下，这将比特安全性从 ML-KEM 的 140 比特提高到 153 比特。基于根式扩张的理想格可用随 k 线性增长的元素表示，而 M-LWE/LWR 关于 k 呈二次增长，因而可缓解矩阵生成成本。与较小格相比，这仍会增大密钥和密文并影响整体性能；但为增强对实际达到 128 比特安全等级的信心，我们认为该代价可以接受。

(u)Saber Saber [DKRV18] 是首个完全基于（模）LWR 的 KEM。它于 2016 年提交 NIST PQC 竞赛并进入第三轮 [DKR⁺20]。完全依赖 LWR 意味着无需从误差分布中采样。Saber 以简洁高效为目标：所有参数集均固定采用环 $\mathbf{Z}_q[x]/(x^{256} + 1)$ 和 $q = 2^{13}$ ，仅改变模维数、密文模数和私钥分布。二幂模数 q 的算术可直接实现，无需模约化或拒绝采样。NIST 提案定义 LightSaber、Saber 和 FireSaber 三个参数集，分别对应 NIST 安全等级 1、3、5；私钥从 $\eta = 3, 4, 5$ 的二项分布 β_η 中选取。选择该 q 的缺点是 $\mathbf{Z}_q$ 原生不支持 NTT，因此早期实现依赖 Toom–Cook。不过，将其嵌入模数经适当选择（比特长度约为 q 的两倍）的更大环后，仍可用 NTT 实现多项式乘法，尽管效率不及 ML-KEM 等方案。

Uniform Saber (uSaber) [DKR⁺20, 第 A.2 节] 是 Saber 第三轮提交中提出、但不属于主提案的变体。主要差异是秘密向量 $\mathbf{s}$ 和 $\mathbf{s}'$ 从均匀分布 $\mathcal{U}_2$ 而非二项分布 β_η 中采样。因此，秘密系数位于 $[-2, 1]$ ，而非 Saber 的 $[-\eta, \eta]$ ，每个系数恰可用 2 比特表示。

uSaber 的主要优点是简化了密钥生成和加密所需的秘密生成过程，因而也简化了 KEM 的封装和解封装。它以显著更少的（伪）随机比特获得相近的秘密方差和安全等级。例如，uSaber 每个秘密系数只需 2 个随机比特，而标准 Saber 的二项采样需要 6 至 10 比特。这使哈希更快，同时降低计算开销与攻击面。哈希是高度非线性的过程，因此该优势对掩码实现尤其重要。

此外，uSaber 不再需要计算二项分布。这一简化使实现更直接，尤其是在掩码场景下：非

线性的二项采样会显著增加复杂度。将其移除既提高效率，又缩小攻击面，能使设计更安全、更实用。

Mithril 遵循 uSaber 的设计原则并将其扩展到 RR-LWR 场景。小均匀分布有利于简洁、安全的实现，包括侧信道防护。采用 RR-LWR 后，通过缩小矩阵（更准确地说，以一个环元素取代矩阵），Mithril 比 uSaber 更高效，也可更细粒度地提高安全等级。Mithril 使用 $n = 128$ 而非 $n = 256$ ，并结合 $k = 5$ ，达到严格高于 uSaber 的 128 比特安全等级。

Scabbard 套件：Florete、Espada 与 Sable 在 LWR 类方案的乘法例程、密码分析和参数选择等改进基础上，NIST 标准化期间研究者们提出了由三个 KEM 构成的 Scabbard 套件 [MKKV21, KNK⁺25]。三个方案分别针对速度、面积需求、内存占用和并行实现等不同方面改进现有水平。

第一个方案 Florete 是 Ring-LWR 方案。Saber 在 $q = 2^{13}$ 的 $\mathbf{Z}_q$ 上采用模秩 $k = 3$ 和多项式次数 $n = 256$ ；Florete 则在 $q = 2^{10}$ 时采用环 $\mathbf{Z}_q[x]/(x^{768} - x^{384} + 1)$ 定义的理想格，并固定 $k = 1$ 。为达到 Saber 的 192 比特安全等级，它将 $\mathbf{A}$ 从 3×3 矩阵缩减为单个环元素，消除了矩阵-向量乘法。生成 $\mathbf{A}$ 所需伪随机字节更少，可缓解尤其是软件中的性能瓶颈；同时可完全复用 Saber 的乘法例程。缺点是其安全性比 Saber 低约 15 比特 [MKKV21, 表 2]。后续工作 [KNK⁺25] 分别采用次数 512 和 1024 的环，以类似方法匹配 LightSaber 与 FireSaber 的安全等级。Florete 虽与 Mithril 一样使用环，但其环无法扩展到不同安全等级，目标也仅为 192 比特安全性。

第二个方案 Espada 通过提高算法并行度来改善效率。其核心是将含 256 个系数的大多项式乘法分解为多个各含 64 个系数的小乘法。Toom-Cook 或 Karatsuba 虽也能把 256×256 乘法化为 64×64 乘法，但并行度不及原生小乘法，且其求值和插值阶段会增大乘法器。令底层格的原生多项式次数为 $n = 64$ ，可降低基本乘法例程在硬件 [MTK⁺20] 和微控制器上的实现成本，并提高参数选择粒度。为达到 128 比特安全等级，模秩 k 从 3 增至 12（其他安全等级类似）。所需 64×64 乘法总数虽多于 Saber，但各乘法相互独立，更易并行。将 n 从 256 降至 64 还要求每个系数承载 $B = 4$ 比特，这会影响失败概率，并最终要求把 q 增至 15 比特。Mithril 与 Espada 的区别是前者为环基 KEM；它同样将 n 从 256 降至 128，但无需每个系数承载多个比特。Mithril 选择 $B = 1$ ，避免对失败概率产生影响。

最后一个方案 Sable 最接近 Saber，但改变了私钥分布。LightSaber、Saber 和 FireSaber 分别取 $\eta = 5, 4, 3$ ，从区间 $[-\eta, \eta]$ 上的中心二项分布 β_η 选择私钥系数；Sable 则在所有参数集中固定 $\eta = 1$ ，使方案切换更顺畅。此时每个私钥系数只需 2 比特，与 uSaber 的存储需求相同，并有利于掩码实现。总体而言，该改变降低了二项乘法器的复杂度；若按 p, q 的最大值实现，还可统一各参数集的乘法器设计。为弥补安全性损失， p 与 q 相应减小。论文指出，Saber 在选择 p 和 q 时把舍入误差建模为连续而非离散均匀分布，因而估计有误。LightSable、Sable 和 FireSable 均采用离散均匀分布，参数选择与安全估计更准确。Mithril 同样追求较小私钥系数，但允许比 Sable 更大的私钥区间，并仅采用均匀分布，从而简化采样并更高效地利用每个采样比特。

SMAUG 及相关方案 SMAUG-T [HCCSS24] 融合了 SMAUG [CCHY24] 与 TiGER [PJP⁺22]，是最新的 LWR 类提案，并入选韩国 PQC 标准化。其密钥生成归约

到 LWE，而封装/解封装依赖 LWR。生成临时封装秘密时采用稀疏中心二项分布：在 $0 \leq r < 1/2$ 时，以 $1 - 2r$ 的概率取 0，以 r 的概率分别取 -1 和 1 ； r 取 $1/8$ 或 $3/16$ ，分母特意选为二幂以方便实现。由于部分采样器并非常数时间实现，该设计在 v2.0 至 v4.0 间曾更新，因此必须谨慎实现。稀疏秘密可减少解密失败并提高 KEM 效率。v3.0 又加入 TiMER 参数集，利用 D2 误差协调机制进一步降低解密失败概率；它把每个消息比特编码到两个多项式系数中，以增加编解码开销为代价降低失败概率。

Mithril 的密钥生成不采用 LWE 假设，私钥也不采用稀疏分布。我们的目标是降低实现复杂度，尤其是与计时攻击及其他侧信道攻击有关的复杂度。稀疏秘密虽可能带来性能优势，Mithril 仍采取保守设计，将安全性置于首位。

3 预备知识

本节介绍理解本文所需的全部背景知识。

3.1 符号约定

环元素 整数环记为 $\mathbf{Z}$ ；对于正整数 q ， $\mathbf{Z}_q$ 表示商环 $\mathbf{Z}/q\mathbf{Z}$ 。给定正整数 m ，第 m 个分圆多项式 (cyclotomic polynomial) 记为 $\Phi_m(x)$ ，定义为以所有本原 m 次单位根为根的唯一首一多项式。具体而言，

$$\Phi_m(x) = \prod_{\substack{k=1 \\ \gcd(k,m)=1}}^m \left(x - e^{2\pi i \frac{k}{m}} \right),$$

其中满足 $\gcd(k, m) = 1$ 的 $e^{2\pi i k/m}$ 正是本原 m 次单位根。可以证明 $\Phi_m(x)$ 的系数均为整数；由定义可知 $\deg(\Phi_m(x)) = \varphi(m)$ ，其中 $\varphi(m)$ 为 Euler 函数 (Euler's totient function)，即小于 m 且与 m 互素的正整数个数。当 $m = 2n$ 且 n 为 2 的幂时，众所周知 $\Phi_m(x) = x^n + 1$ 。多项式商环记为 $\mathbf{R}_q = \mathbf{Z}_q[x]/(f(x))$ ，其中 $f(x) \in \mathbf{Z}[x]$ 为首一多项式。

设 n 为 2 的幂， k 为正整数。定义分圆环 $\mathbf{R}_{q,n} = \mathbf{Z}_q[y]/(y^n + 1)$ 以及

$$\mathbf{S}_{q,n,k} = \mathbf{R}_{q,n}[x]/(x^k - y - 2)$$

并将后者视为 $\mathbf{R}_{q,n}$ 的 k 次根式扩张。 $\mathbf{S}_{q,n,k}$ 中的元素表示为多项式 $\mathbf{g} = \sum_{i=0}^{k-1} \mathbf{g}_i x^i$ ，其系数 $\mathbf{g}_i \in \mathbf{R}_{q,n}$ 。普通小写字母表示 $\mathbf{Z}$ 或 $\mathbf{Z}_q$ 中的元素；粗体小写字母表示 $\mathbf{R}_{q,n}$ 或 $\mathbf{S}_{q,n,k}$ 中的元素 (多项式或环元素)；粗体大写字母表示矩阵。

模表示 对于偶数 α 和 $r \in \mathbf{Z}$ ，记 $r' = r \bmod \alpha$ 为区间 $[-\alpha/2, \alpha/2)$ 中满足 $r' \equiv r \pmod{\alpha}$ 的唯一代表元；该区间包含 $-\alpha/2$ ，但不包含 $\alpha/2$ 。当 α 为 2 的幂时，这等价于提取 r 的最低 $\log_2(\alpha)$ 个比特。除非另有说明，所有变量均假定表示在有符号区间内。在具有 32 位或 64 位字长的现代 CPU 上实现该操作时，若丢弃高位，须注意符号扩展。此操作按系数推广到 $\mathbf{S}_{q,n,k}$ 的元素。

范数 对于环元素 $\mathbf{g} = \sum_{i=0}^{k-1} \mathbf{g}_i x^i \in \mathbf{S}_{q,n,k}$ 以及 $\mathbf{g}_i = \sum_{j=0}^{n-1} \mathbf{g}_{i,j} y^j \in \mathbf{R}_{q,n}$ ，定义 $\|\mathbf{g}\|_\infty = \max_{i,j} |\mathbf{g}_{i,j} \bmod q|$ 。

采样分布 用 $D_{\mathbf{x}}(R)$ 表示从环 R 中采样变量 $\mathbf{x}$ 所用的分布。若由种子 σ 确定性地完成采样, 则记为 $D_{\mathbf{x}}(R; \sigma)$ 。若使用多个种子 $\sigma_1, \dots, \sigma_m$, 则记为 $D_{\mathbf{x}}(R; \sigma_1, \dots, \sigma_m)$; 采样前将其串接为 $\sigma = \sigma_1 \mid \dots \mid \sigma_m$ 。例如, 可用这种方式向各个种子追加环索引。

哈希函数 算法使用输出长度可变 (可扩展) 的哈希函数 $H(\text{input}, \text{outlen})$, 其中 outlen 以字节为单位。每次调用均单独规定 H 的输出长度。安全性证明要求 H 表现为随机预言机, 并具有 $\lambda/2$ 比特抗碰撞性和 λ 比特抗原像性, 其中 λ 为目标安全等级。 H 的具体实例将与最终参数集一同规定。

3.2 格基密码学

要理解舍入学习 (Learning with Rounding, LWR) 的理论与实践, 需要先介绍误差学习 (Learning with Errors, LWE) 问题 [Reg05], 以及更常用的结构化变体: 环上误差学习 (Ring-LWE) [LPR10] 和模上误差学习 (Module-LWE) [BGV12, LS15]。感兴趣的读者可参阅有关 LWE 的综述 [Reg10, Pei16]。

3.2.1 误差学习

设模秩 d 为正整数, χ 为 $\mathbf{R}$ 上的一个分布。设 $s \in \mathbf{R}_q^d$ 为秘密向量, 定义分布 $A_{s, \chi}$ 为

$$A_{s, \chi} = (a, b = \langle a, s \rangle + e \bmod q) \in \mathbf{R}_q^d \times \mathbf{R}_q$$

其中 $a \leftarrow \mathcal{U}(\mathbf{R}_q^d)$ 均匀随机采样, $e \leftarrow \chi$ 为误差项。

搜索 LWE 问题要求在获得从 $A_{s, \chi}$ 独立抽取的多项式个样本后恢复秘密 s ; 判定 LWE 问题要求区分从 $A_{s, \chi}$ 抽取的样本与从 $\mathbf{R}_q^d \times \mathbf{R}_q$ 均匀抽取的样本。实际中通常取 $\mathbf{R}_{q, n} = \mathbf{Z}_q[x]/(f(x))$, 其中 $f(x)$ 为分圆多项式

$$f(x) = \Phi_{2n}(x) = x^n + 1$$

且 n 为 2 的幂。在这一 Ring-LWE 变体中, 所有运算均在环 $\mathbf{R}_q$ 内进行。一个 Ring-LWE 样本包含相当于 n 个普通 LWE (Plain-LWE) 样本的信息, 同时可显著缩小密码密钥。分圆多项式的选择还使多项式乘法能够借助仅涉及整数运算的傅里叶变换变体——数论变换 (Number Theoretic Transform, NTT) [Pol71]——得到高效计算。

Module-LWE 问题 [BGV12, LS15] 同时推广了 Plain-LWE 与 Ring-LWE, 提供一种可在安全性和效率之间权衡的灵活中间结构。在 Module-LWE 中, 秘密为多项式向量, 一个样本形如 $(\mathbf{a}, b)$, 其中 $b = \langle \mathbf{a}, \mathbf{s} \rangle + e$ 。因此, Module-LWE 是 Plain-LWE 和 Ring-LWE 的推广: 令秩 $d = 1$ 即得到 Ring-LWE; 令底层环为 $\mathbf{Z}$ 且秩 $d = n$ 即得到 Plain-LWE。此外, 在其他参数给定时, 只需调整模秩即可调节 Module-LWE 的困难性, 即方案的安全性。

3.2.2 舍入学习

舍入学习由文献 [BPR12] 提出, 可视为 LWE 的一种去随机化技术。在 LWE 中计算 $\langle a, s \rangle + e \bmod q$, 即在内积上加上误差项 e (模 q)。LWR 不采用对误差的加法, 而是把内积缩放并舍

入到较小的群 $\mathbf{Z}_p$ 中。具体而言, 该舍入将 $\mathbf{Z}_q$ 中的元素划分为 p 个连续区间, 并将每个元素映射到其区间在 $\mathbf{Z}_p$ 中的索引。必须有 $p < q$, 否则不会引入误差。舍入后的内积 $\lfloor \frac{p}{q} \langle a, s \rangle \rfloor$ 记为

$$\lfloor \langle a, s \rangle \rfloor_p = \left\lfloor \frac{p}{q} \langle a, s \rangle \right\rfloor \in \mathbf{Z}_p.$$

这里用 $[0, q[$ (相应地用 $[0, p[$) 表示 $\mathbf{Z}_q$ (相应地表示 $\mathbf{Z}_p$)。当 $p \mid q$ 时, $\mathbf{Z}_q$ 上的均匀分布经舍入后仍为 $\mathbf{Z}_p$ 上的均匀分布。舍入运算可逐系数推广到向量和多项式。与 LWE 的环变体和模变体类似, 也可定义 LWR 的环变体和模变体。

舍入学习问题的判定版本定义如下。

Definition 1 (判定舍入学习, 基于文献 [BPR12, 定义 3.1]). 设 $n \geq 1$, 模数 $q > p \geq 2$ 均为整数, $\mathbf{R}_q = \mathbf{Z}_q[x]/f(x)$ 如上定义。

- (**Plain-LWR**): 对于向量 $s \in \mathbf{Z}_q^n$, 定义 LWR 分布 L_s 为 $\mathbf{Z}_q^n \times \mathbf{Z}_p$ 上如下产生的分布: 均匀随机选择向量 $a \leftarrow \mathbf{Z}_q^n$, 输出 $(a, b = \lfloor \langle a, s \rangle \rfloor_p)$ 。
- (**Ring-LWR**): 对于多项式 $s \in \mathbf{R}_q$, 定义 Ring-LWR 分布 L_s 为 $\mathbf{R}_q \times \mathbf{R}_p$ 上如下产生的分布: 均匀随机选择 $a \leftarrow \mathbf{R}_q$, 输出 $(a, b = \lfloor a \cdot s \rfloor_p)$ 。
- (**Module-LWR**): 固定模秩 $d > 1$ 。对于 $s \in \mathbf{R}_q^d$, 定义 Module-LWR 分布 L_s 为 $\mathbf{R}_q^d \times \mathbf{R}_p$ 上如下产生的分布: 均匀随机选择 $a \leftarrow \mathbf{R}_q^d$, 输出 $(a, \lfloor \langle a, s \rangle \rfloor_p)$ 。

判定 (环/模) LWR_{n,q,p} 问题要求以关于 n 不可忽略的优势, 区分多项式个独立样本 $(a_i, b_i) \leftarrow L_s$ 与相同数量、分别从 $\mathbf{Z}_q^n \times \mathbf{Z}_p$ 、 $\mathbf{R}_q \times \mathbf{R}_p$ 或 $\mathbf{R}_q^d \times \mathbf{R}_p$ 中均匀独立抽取的样本。

3.2.3 舍入学习的安全性

证明 LWR 安全性的总体思路是说明其安全性在统计意义上接近于求解判定 LWE (即与随机 $\mathbf{u}$ 区分) 的安全性 [AKPW13]:

$$(a, \lfloor \langle a, s \rangle \rfloor_p) \stackrel{\text{stat}}{\approx} (a, \langle a, s \rangle + e) \stackrel{\text{comp}}{\approx} (a, \mathbf{u}).$$

文献 [BPR12] 首次研究了 LWR 的安全性, 并给出如下结果。

Theorem 1 (到 LWE 的归约 [BPR12, 定理 3.2]). 设 χ_n 是由安全参数 n 索引、定义在 $\mathbf{Z}$ 上且可高效采样的任意概率分布族, 并且其范围受 $\beta(n)$ 约束 (即 $\Pr_{x \leftarrow \chi}[|x| > \beta] \leq \text{negl}(n)$)。设 $q \geq p \cdot \beta \cdot n^{\omega(1)}$ 。则对于秘密 $s \in \mathbf{Z}_q^n$ 上的任意分布, 求解判定 $\text{LWR}_{n,q,p}$ 至少与在相同秘密分布下求解判定 $\text{LWE}_{n,q,\chi}$ 一样困难。

定理 1 要求模数 $q \geq p \cdot \beta \cdot n^{\omega(1)}$ 为超多项式量级。文献 [AKPW13] 对此加以改进, 给出了适用于更广参数范围的安全性归约: 所用模数 q 只需为多项式量级, 而非超多项式量级 [AKPW13, 定理 4.1 与推论 4.2]。具体地, 假定参数为 n', m, q, β 的 LWE 是困难的, 只要

$$n \geq \frac{\log(q)}{\log(2\gamma)} \cdot n' \quad , \quad q \geq \gamma(nm\beta p)$$

对某个 $\gamma \geq 1$ 成立，作者便证明了参数为 n, m, q, p （其中 m 为样本数）的 LWR 的困难性。为使归约成立，模数 q 必须远大于 LWR 样本数 m 。因此，当允许攻击者选择样本数（而不是预先固定）时， q 又成为超多项式量级。模数 q 还必须为素数；否则，上述不等式须对 q 的最大素因子 $q_{\max}$ 成立。此外，该结果要求 q 与 $q/q_{\max}$ 互素，这意味着模数 q 为 2 的幂时结果不成立。

文献 [BGM⁺16] 的结果消除了这一限制。首先，模数 q 的界改进为

$$q \geq 2m\beta p$$

其次，该结果对 q 不施加任何数论限制。特别地，当 q 为 2 的幂时，该结果仍然成立 [BGM⁺16, 定理 1 与定理 3]。

Theorem 2 (LWR 的单向性 [BGM⁺16, 定理 1]). 设 p, q, n, m, β 为满足 $q > 2p\beta$ 的整数。对于任意算法 $Learn$,

$$\Pr_{A,s,e} [Learn(A, [As + e]_p) = s] \geq \frac{\Pr_{A,s} [Learn(A, [As]_p) = s]^2}{(1 + 2p\beta/q)^m},$$

其中 $A \leftarrow \mathbf{Z}_q^{m \times n}$ ；噪声 e 的所有 m 个坐标相互独立，每个坐标均受 β 约束且平衡； s 从支撑集为 $\mathbf{Z}_q^{n*}$ 的任意分布中选取。

该结果证明了样本数 m 有界时 LWR 的困难性。近期文献 [NR23] 研究了无界情形，并指出在这一情形下使用从 LWE 到 LWR 的“逐点”归约存在障碍。该工作的主要结果及相关猜想要求模数 q 为素数 [NR23, 定理 1 与猜想 1]，而本设计中的 q 并非素数。应当指出，文献 [NR23] 的结果只是表明在无界样本情形下证明安全性需要新的思路，并不意味着 LWR 是容易求解的。

3.3 作为结构化模的环

在 Module-LWR 中，单个样本形如 $[\langle \mathbf{A}, \mathbf{s} \rangle]_p$ ，其中 $\mathbf{A}, \mathbf{s} \in \mathbf{R}_{q,n}^k$ ， k 为模秩。样本的选择使 $\mathbf{A}$ 服从均匀分布，而 $\mathbf{s}$ 和 $\mathbf{e}$ 从“小”分布中生成。若选取 m 个样本，并假定环结构和模结构均无法被利用，则其安全等级等价于一个矩阵大小为 $mn \times kn$ 的 Plain-LWR 问题。令 $m = k$ 即得到典型 KEM 的情形，此时舍入学习问题由方阵定义。为简便起见，下文均假定 $m = k$ 。

Module-LWR 的主要优点是可以任意选择 k 。例如，Saber 选择 $k \in \{2, 3, 4\}$ 。相比之下，要达到相同安全等级，基于分圆环的 Ring-LWR 需从 $R_{q,kn}$ 中采样，因而必须将 k 限制为 2 的幂，不能选择 $k = 3$ 。另一方面，Ring-LWR 的优点是样本元素 $\mathbf{A}$ 只包含 kn 个不同的 $\mathbf{Z}_q$ 元素（即单个 $\mathbf{A} \in R_{q,kn}$ ），而 Module-LWR 包含 k^2n 个（即 $\mathbf{R}_{q,n}^k$ 中元素的 k 个样本）。Module-LWR 随 k 二次增长，效率低于 Ring-LWR；但其在 k 取值上的更大灵活性使具体参数集更易实例化。

众所周知，Ring-LWR 可对应于（或视为）一个结构化 Module-LWR 实例。对于每个满足 $k \mid n$ 且为 2 的幂的 k ，存在同构

$$\mathbf{R}_{q,n} = \mathbf{Z}_q[x]/(x^n + 1) \cong \left(\mathbf{Z}_q[y]/(y^{\frac{n}{k}} + 1) \right) [x]/(x^k - y) \quad (2)$$

映射为

$$s(x) = \sum_{i=0}^n s_i x^i \mapsto \sum_{j=0}^{k-1} s_j(y) \cdot x^j, \quad \text{其中} \quad s_j(y) = \sum_{i=0}^{n/k-1} s_{ki+j} \cdot y^i.$$

该映射只是将多项式 s 分成 k 个部分 $s_j(y)$, 每个 $s_j(y)$ 包含次数模 k 等于 j 的系数。例如, 当 $k = 2$ 时, $s_0(y)$ 包含偶数次系数, $s_1(y)$ 包含奇数次系数。对两个元素 $\mathbf{A}, \mathbf{s} \in \mathbf{R}_{q,n}$, 在该同构下有

$$\mathbf{A} \cdot \mathbf{s} = \begin{bmatrix} a_0(y) & ya_1(y) \\ a_1(y) & a_0(y) \end{bmatrix} \begin{bmatrix} s_0(y) \\ s_1(y) \end{bmatrix},$$

其中各项运算均在环 $\mathbf{Z}_q[y]/(y^{\frac{n}{2}} + 1)$ 中进行; 特别地, ya_1 对应于 $a_1(y)$ 系数的反循环旋转。这是因为

$$\begin{aligned} \mathbf{A} \cdot \mathbf{s} &= (a_0(y) + a_1(y)x) \cdot (s_0(y) + s_1(y)x) \\ &= a_0s_0(y) + (a_0s_1(y) + a_1s_0(y))x + a_1s_1(y)x^2 \\ &= a_0s_0(y) + ya_1s_1(y) + (a_0s_1(y) + a_1s_0(y))x \bmod (x^2 - y). \end{aligned}$$

当 $k = 4$ 时, 则有

$$\mathbf{A} \cdot \mathbf{s} = \begin{bmatrix} a_0(y) & y \cdot a_3(y) & y \cdot a_2(y) & y \cdot a_1(y) \\ a_1(y) & a_0(y) & y \cdot a_3(y) & y \cdot a_2(y) \\ a_2(y) & a_1(y) & a_0(y) & y \cdot a_3(y) \\ a_3(y) & a_2(y) & a_1(y) & a_0(y) \end{bmatrix} \begin{bmatrix} s_0(y) \\ s_1(y) \\ s_2(y) \\ s_3(y) \end{bmatrix}, \quad (3)$$

此时运算在 $\mathbf{Z}_q[y]/(y^{\frac{n}{4}} + 1)$ 中进行。更一般地, 容易看出 $\mathbf{A} \cdot \mathbf{s}$ 对应于一个 $k \times k$ 循环矩阵与向量的乘法, 其中矩阵右上三角部分乘以 y 。作为比较, 当 $k = 4$ 时, Module-LWR 矩阵乘法的结构为

$$\begin{bmatrix} a_{0,0}(y) & a_{0,1}(y) & a_{0,2}(y) & a_{0,3}(y) \\ a_{1,0}(y) & a_{1,1}(y) & a_{1,2}(y) & a_{1,3}(y) \\ a_{2,0}(y) & a_{2,1}(y) & a_{2,2}(y) & a_{2,3}(y) \\ a_{3,0}(y) & a_{3,1}(y) & a_{3,2}(y) & a_{3,3}(y) \end{bmatrix} \begin{bmatrix} s_0(y) \\ s_1(y) \\ s_2(y) \\ s_3(y) \end{bmatrix},$$

其中算术同样在环 $\mathbf{Z}_q[y]/(y^{\frac{n}{4}} + 1)$ 中进行。

总而言之, 对于任意 $k \mid n$, 都可将一个分圆 Ring-LWR 样本视为秩为 k 的结构化 Module-LWR 样本, 其中

$$a_{i,j}(y) = \begin{cases} \text{均匀随机}, & j = 0, \\ y \cdot a_{k+i-j,0}(y), & 0 \leq i < j < k, \\ a_{i-j,0}(y), & 0 < j \leq i < k. \end{cases} \quad (4)$$

注意, 上述重新解释只是形式上的, 不会影响安全性。在式 (2) 的环同构下, 一个分圆 Ring-LWR 样本可以双射到式 (4) 所述的秩 k 结构化 Module-LWR 实例。任何针对结构化 Module-LWR 的区分器或秘密恢复算法, 与逆同构复合后即可得到一个针对 Ring-LWR 的算法, 其优势相同、运行时间也基本相同。因此, 把分圆 Ring-LWR 样本视为秩 k 的结构化 Module-LWR 样本, 并不会改变底层困难性假设或具体估计。这只是同一分布的另一种表示 (相同论证也适用于 LWE)。

3.4 根式环 LWR

第 3.3 节的观察是以不同方式解释并定义 Ring-LWR 和 Module-LWR 类方案的新环与参数的关键。

我们不采用 2 的幂次分圆多项式 $f(x) = x^n + 1$ ，而是在 n 为 2 的幂、 k 为严格正整数时选择 $f(x) = (x^k - 2)^n + 1$ 。可以证明 f 不可约 [BRVW26, 引理 1]，因此 $\mathbf{Q}[x]/(f(x))$ 是一个数域。于是可以针对该 $f(x)$ 定义 Ring-LWR 问题，其困难性基于典范嵌入下相应理想格中的格问题 [LPR10]。乍看之下，改变多项式似乎会失去 2 的幂次分圆多项式所带来的优势；但存在如下简单的环同构：

$$S_{q,n,k} := \mathbf{Z}_q[x]/((x^k - 2)^n + 1) \cong (\mathbf{Z}_q[y]/(y^n + 1)[x])/(x^k - y - 2), \quad (5)$$

$$x^k \mapsto y + 2.$$

虽然 $S_{q,n,k}$ 本身不是分圆环，但它含有一个较大的 2 的幂次分圆子环，仍可在其中采用 NTT 等算术优化。因而，由 $f(x)$ 定义的域是第 $2n$ 个分圆域的 k 次扩张；扩张次数 k 的作用类似于模秩，使底层格问题的困难性比要求 n 必须为 2 的幂的分圆环更易调节。由于多项式 $x^k - y - 2$ 定义了第 $2n$ 个分圆域的 k 次根式扩张，在舍入学习场景中将这类环称为根式 Ring-LWR (RR-LWR)。

需要注意，即使考虑一般的第 m 个分圆域，而不只考虑 2 的幂 n ，仍无法构造既含有较大 2 的幂次分圆子域、又具有给定次数的分圆域。第 m 个分圆域的次数为 Euler 函数 $\varphi(m)$ ，因此可能的扩张次数受限于这类整数；若出于效率考虑还要求较大的 2 的幂次子域，限制会更严格。例如，不存在满足 $\varphi(m) = 3$ 或 $\varphi(m) = 5$ 的整数 m ，所以不存在作为二幂分圆域 3 次或 5 次扩张的分圆域。采用上述 $f(x)$ 后，只需令 $k = 3$ 或 $k = 5$ 即可直接得到这样的域。

与一般模结构相比，上述构造通过施加环结构，降低了相对于 Module-LWR 的计算复杂度和随机性需求。具体而言，矩阵 $\mathbf{A}$ 由分圆环中的 k 个而非 k^2 个均匀随机元素组成。更具体地，对 $\mathbf{A}, \mathbf{s} \in S_{q,n,k}$ ，写作 $\mathbf{A} = \sum_{i=0}^{k-1} a_i(y)x^i$ 以及 $\mathbf{s} = \sum_{i=0}^{k-1} s_i(y)x^i$ ，则多项式乘法 $\mathbf{A} \cdot \mathbf{s}$ 对应于矩阵乘法

$$\begin{bmatrix} a_0(y) & (y+2) \cdot a_{k-1}(y) & (y+2) \cdot a_{k-2}(y) & \cdots & (y+2) \cdot a_1(y) \\ a_1(y) & a_0(y) & (y+2) \cdot a_{k-1}(y) & \cdots & (y+2) \cdot a_2(y) \\ \vdots & \vdots & \vdots & & \vdots \\ a_{k-1}(y) & a_{k-2}(y) & a_{k-3}(y) & \cdots & a_0(y) \end{bmatrix} \begin{bmatrix} s_0(y) \\ s_1(y) \\ s_2(y) \\ \vdots \\ s_{k-1}(y) \end{bmatrix}. \quad (6)$$

该矩阵乘法中的全部运算都在分圆环 $\mathbf{Z}_q[y]/(y^n + 1)$ 中进行。与式 (3) 的 Ring-LWR 矩阵结构相比，这里实质上只是将乘法因子 y 替换为 $y + 2$ 。

在不同于分圆环的数环上实例化 Ring-LWR 时必须谨慎。我们将在安全性部分证明，本实例确实是 Ring-LWR 的安全实例。

	Mithril-128	Mithril-256	Mithril-512
n	128	128	128
k	5	9	17
ℓ	1	2	4
ϵ_q	13	13	13
ϵ_p	11	11	11
ϵ_t	3	3	8
D_A	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_{\epsilon_q}$	$\mathcal{U}_{\epsilon_q}$
D_s	$\mathcal{U}_2$	$\mathcal{U}_2$	$\mathcal{U}_2$
KEM 私钥 (字节)	1 136	2 000	3 728
KEM 公钥 (字节)	944	1 648	3 056
KEM 密文 (字节)	928	1 680	3 504
共享秘密 (字节)	16	32	64
失败概率 ($\log_2$)	-603	-280	-230
Core-SVP (经典, 比特)	129	257	526
Core-SVP (量子, 比特)	117	233	478

表 2: Mithril 三个参数集的概览。

4 算法描述

本节描述构造 Mithril KEM 所需的全部组件。Mithril.KEM 通过带隐式拒绝的 Fujisaki–Okamoto 变换由 Mithril.PKE 导出。Mithril.PKE 包含 KeyGen、Enc 和 Dec 三个算法；Mithril.KEM 包含 KeyGen、Encaps 和 Decaps，完整算法见图 1。

参数集与安全等级。 算法按照提交要求参数化，以支持多个安全等级。具体规定三个参数集，分别以 128、256 和 512 比特经典安全强度为目标；相应的抗量子安全强度估计分别不低于 80、128 和 256 比特。

各参数集实例化相同的算法结构，区别仅在于决定底层格维数和模数大小的参数选择。该统一设计使所有安全等级可采用一致的实现框架，同时可通过参数选择细粒度调节安全性。完整参数见表 2。

环的选择。 本方案在如下根式环上运算：

$$\mathbf{S}_{q,n,k} = (\mathbb{Z}_q[y]/(y^n + 1))[x]/(x^k - y - 2), \quad (7)$$

其中 q 和 n 为 2 的幂， k 为任意正整数。Octarine 也使用同一环。存在直接同构 $\mathbf{S}_{q,n,k} \cong \mathbb{Z}_q[x]/((x^k - 2)^n + 1)$ [BRVW26, 第 4 节]。本方案在根式环系数表示下使用 RR-LWR；在该同构下，它对应于相关数域上的基系数 Ring-LWR 实例。环 $\mathbf{S}_{q,n,k}$ 是第 $2n$ 个分圆基环的 k 次根式扩张，因此可按需采用 NTT 或其他多项式乘法例程等标准实现技术。由 k 个基环分量构成的紧凑表示取代了秩 k 模构造中随秩二次增长的公开矩阵表示。

所有参数集均固定环模数 $q = 2^{13}$ 和基环次数 $n = 128$ ，通过改变根式扩张次数 k ，从而改

$(pk, sk) \leftarrow \text{Mithril.PKE.KeyGen}()$	
1: $\text{seed}_A, \text{seed}_s \leftarrow \{0, 1\}^{512}$	
2: $A \leftarrow D_A(S_{q,n,k}; \text{seed}_A)$	
3: $s \leftarrow D_s(S_{q,n,k}; \text{seed}_s)$	
4: $b = \lfloor \frac{p}{q} As \rfloor \in \mathbf{S}_{p,n,k}$	
5: 返回 $(pk = (\text{seed}_A, b), sk = s)$	
$c \leftarrow \text{Mithril.PKE.Enc}(pk = (\text{seed}_A, b), m)$	
1: $\text{seed}_{s'} \leftarrow \{0, 1\}^{512}$	
2: $s' \leftarrow D_s(S_{q,n,k}; \text{seed}_{s'})$	
3: $A \leftarrow D_A(S_{q,n,k}; \text{seed}_A)$	
4: $b' = \lfloor \frac{p}{q} \cdot (As') \rfloor \in \mathbf{S}_{p,n,k}$	
5: $v' = (bs')^{[k-\ell:k-1]} + \frac{q}{2 \cdot p} \in \mathbf{R}_{p,n}^\ell$	▷ 选取环元素最高的 ℓ 个系数
6: $c_m = \lfloor \frac{t}{p} \cdot ((v' + \frac{p}{2} \cdot m) \bmod p) \rfloor \in \mathbf{R}_{t,n}^\ell$	
7: 返回 $c = (c_m, b')$	
$m' \leftarrow \text{Mithril.PKE.Dec}(sk = s, c = (c_m, b'))$	
1: $v = (bs')^{[k-\ell:k-1]} \in \mathbf{R}_{p,n}^\ell$	▷ 选取环元素最高的 ℓ 个系数
2: $m' = \lfloor \frac{2}{p} \cdot ((v - \frac{p}{t} \cdot c_m + (\frac{p}{2 \cdot t} - \frac{q}{2 \cdot p})) \bmod p) \rfloor \in \mathbf{R}_{2,n}^\ell$	
3: 返回 m'	
$(pk, sk) \leftarrow \text{Mithril.KEM.KeyGen}()$	
1: $(pk_{\text{pke}}, sk_{\text{pke}}) \leftarrow \text{Mithril.PKE.KeyGen}$	
2: $z \leftarrow \{0, 1\}^{\ell n}$	▷ 秘密回退值
3: $hpk \leftarrow \mathcal{F}_{H_F}(pk_{\text{pke}}, \lambda/8)$	▷ 公钥哈希值
4: 返回 $(pk = pk_{\text{pke}}, sk = (sk_{\text{pke}}, pk_{\text{pke}}, hpk, z))$	
$(c, ss) \leftarrow \text{Mithril.KEM.Encaps}(pk)$	
1: $m \leftarrow \{0, 1\}^{\ell n}$	▷ 编码/映射为 $\mathbf{R}_{2,n}^\ell$ 中的消息
2: $hpk \leftarrow \mathcal{F}_{H_F}(pk, \lambda/8)$	
3: $(ss, \text{seed}_{s'}) \leftarrow \mathcal{G}_{H_G}(hpk, m, \lambda/8 + 64)$	▷ 导出密钥材料和加密种子
4: $c \leftarrow \text{Mithril.PKE.Enc}(pk, m; \text{seed}_{s'})$	▷ 使用显式随机量 $\text{seed}_{s'}$ 进行加密
5: 返回 (c, ss)	
$ss \leftarrow \text{Mithril.KEM.Decaps}(sk, c)$	
1: $m' \leftarrow \text{Mithril.PKE.Dec}(sk_{\text{pke}}, c)$	
2: $(ss', \text{seed}'_{s'}) \leftarrow \mathcal{G}_{H_G}(hpk, m', \lambda/8 + 64)$	▷ 重新计算密钥材料和加密种子
3: $c' \leftarrow \text{Mithril.PKE.Enc}(pk, m'; \text{seed}'_{s'})$	▷ 执行确定性重新加密
4: $\bar{ss} \leftarrow \mathcal{H}_{H_H}(c, z, \lambda/8)$	
5: if $c = c'$ then	▷ 以常数时间实现
6: $ss \leftarrow ss'$	
7: else	
8: $ss \leftarrow \bar{ss}$	
9: end if	
10: 返回 ss	

图 1: Mithril 的 PKE 与 KEM 算法规范。

变总代数次数 nk ，来达到所需安全等级。该环虽对 NTT 友好，但使用 NTT 并非强制。RR-LWR 舍入将系数从模 q 映射到舍入模数 p ，缩放因子为 q/p ，所产生确定性量化误差的绝对值不超过 $q/(2p)$ 。我们选择 p ，使 s 和 s' 均服从 $[-\eta, \eta - 1]$ 上的同一均匀分布，其中 $\eta = q/(2p) = 2$ 。因此，二者的每个系数均恰可用 $\log_2(2\eta) = 2$ 比特表示。

消息空间。 在 Module-LWE 或 Module-LWR 中，消息嵌入为 $\mathbf{R}_{2,n}$ 的元素。该元素含 n 个单比特系数，自然编码 n 比特消息。在 Ring-LWR 中，消息改为 $\mathbf{S}_{2,n,k}$ 的元素，可编码 nk 比特。由于无需完整的 kn 比特消息空间，我们将消息嵌入到一个子空间：其中 ℓ 个系数非零，其余 $k - \ell$ 个系数为 0。因 $n = 128$ ，三个参数集分别选择 $\ell = 1, 2, 4$ ，以保证 $n\ell = \lambda$ 。

为优化性能，选取环元素中最高的 ℓ 个次数系数。这样可尽量减少模 $x^k - (y + 2)$ 的约化次数，节省部分乘法；实现中可省略所有与零系数的乘法。更准确地，记 $\mathbf{g}^{[k-\ell:k-1]}$ 表示多项式 $\mathbf{g} = \sum_{i=0}^{k-1} \mathbf{g}_i(y)x^i \in \mathbf{S}_{2,n,k}$ 中 $\mathbf{g}_0(y), \dots, \mathbf{g}_{k-\ell-1}(y) = 0$ 的部分，并将其解释为向量 $\mathbf{v} \in \mathbf{R}_{2,n}^\ell$ ，满足 $\mathbf{v}_i = \mathbf{g}_{k-1-i}$ 。

哈希。 Mithril 需要使用哈希函数，更准确地说，需要使用可扩展输出函数 (eXtensible Output Function, XOF)，以实例化随机预言机，并为密钥材料和临时秘密生成伪随机比特。我们使用函数 $\mathcal{F}$ 、 $\mathcal{G}$ 和 $\mathcal{H}$ 实例化随机预言机。依照惯例，调用哈希函数时会指定以字节为单位的输出长度。在调用底层 XOF 之前，实现使用显式的域编码来区分这些用途。编码后的输入为

$$\text{ARCANE-Mithril-v1} \parallel |\text{label}| \parallel \text{label} \parallel \lambda \parallel k \parallel \text{outlen} \parallel t \parallel |X_1| \parallel X_1 \parallel \dots \parallel |X_t| \parallel X_t,$$

其中，outlen 是以字节为单位的请求输出长度，每个字段长度 $|X_i|$ 均编码为 64 比特小端序整数。固定前缀为 ASCII 字符串 ARCANE-Mithril-v1。标签长度 $|\text{label}|$ 、 k 和字段数量 t 各编码为一个字节； λ 编码为 16 比特小端序整数；outlen 编码为 32 比特小端序整数。三个 KEM 随机预言机用途所对应的标签分别为 H_F、H_G 和 H_H。具体而言：

- 公钥摘要： $hpk \leftarrow \mathcal{F}_{\text{H-F}}(pk_{\text{pke}}, \lambda/8)$ 生成公钥摘要，该摘要可作为私钥的一部分存储。
- 确定性加密： $(ss, \text{seed}_{s'}) \leftarrow \mathcal{G}_{\text{H-G}}(hpk, m, \lambda/8 + 64)$ 根据公钥摘要和消息生成共享秘密及公钥加密所需的确定性随机量。
- 共享秘密生成： $\bar{s}s \leftarrow \mathcal{H}_{\text{H-H}}(c, z, \lambda/8)$ 生成密文被隐式拒绝时使用的共享秘密。

对于随本规范提交的参考实现和优化实现，所有哈希函数均使用 pseudoXOF 实例化。为与当前先进的优化实现进行公平的性能比较，我们还提供一种基于 SHAKE 的基准测试配置，其中相关哈希函数使用 SHAKE256 实例化。由于 SHAKE256 的容量为 512 比特，最多只能提供 256 比特经典安全性，因此该 SHAKE 配置不用于 512 比特参数集。

我们还需要使用 XOF 生成伪随机比特，以根据种子扩展公私钥材料。该过程用于生成 s 、 $\mathbf{A}$ 和 s' 。公开数据 $\mathbf{A}$ 使用 XOF_A ，秘密数据使用 XOF_S 。这些 XOF 输入采用相同的带长度界定编码，其标签分别为 XOF_A 和 XOF_S 。对于 XOF_A ，输入字段依次为种子、单字节多项式索引和单字节通道索引。对于 XOF_S ，输入字段依次为种子、取值为 0 的索引字节和取值为 0 的通道字节；使用单字节流生成全部 k 个秘密多项式。在所提交实现中， XOF_A 和 XOF_S 均使用 pseudoXOF 实例化。在用于同当前先进优化实现进行公平性能比较的优化配置中， XOF_A 使用 SHAKE128， XOF_S 使用 SHAKE256。

采样分布。 秘密值和公开值从不同的均匀分布中采样。除非另有明确说明，比特和字节寻址均采用小端序。对于较大系数，例如 $\log_2(q) = 13$ 比特的系数，使用域分离独立生成 k 个各含 n 个系数的多项式，从而可并行生成环元素的各多项式。对于系数较小的环元素，例如系数为 2 比特的 s ，每个多项式仅需 $2 \cdot 128/8 = 32$ 字节，因此选择从单一字节流生成整个环元素。

- $D_A(\mathbf{S}_{q,n,k}, \text{seed}_A)$: 使用 XOF 将种子 seed_A 确定性扩展为 $\mathbf{S}_{q,n,k}$ 中的均匀随机元素。生成 $\mathbf{A}$ 共需 k 个含 n 个系数的多项式，每个系数 $\epsilon_q = 13$ 比特。写作 $\mathbf{A} = \sum_{i=0}^{k-1} \mathbf{A}_i(y)$ 、 $\mathbf{A}_i(y) = \sum_{j=0}^{n-1} \alpha_{i,j} y^j$ 。采样分两步：首先从 seed_A 生成字节流，对每个多项式用索引 i 进行域分离，即 $\mathbf{A}_i(y)$ 的字节流为从 seed_A 生成的 $13 \cdot 128/8 = 208$ 字节流 $B_i = \mathbf{G}(\text{seed}_A \parallel i)$ ；然后令 $\alpha_{i,j} = B_i[13j : 13(j+1)]$ ，其中 $B_i[a : b]$ 表示从比特索引 a （含）到 b （不含）的片段。
- $D_s(\mathbf{S}_{q,n,k}, \text{seed}_s)$: 采样一个环元素，其 nk 个整数系数独立均匀取自 $[-\eta, \eta - 1]$ 。生成 $\mathbf{s}$ 共需 k 个含 n 个系数的多项式，每个系数含 $\theta = \log_2(2\eta)$ 比特。写作 $\mathbf{s} = \sum_{i=0}^{k-1} \mathbf{s}_i(y)$ 、 $\mathbf{s}_i(y) = \sum_{j=0}^{n-1} \sigma_{i,j} y^j$ 。首先，XOF $\mathbf{F}$ 使用种子 seed_s 输出字节流 $B = \mathbf{F}(\text{seed}_s)$ ；然后令 $\sigma_{i,j} = B[\theta(in+j) : \theta(in+j+1)]$ ，其中 $B[a : b]$ 的区间约定同上。对种子 seed_s' 生成 $\mathbf{s}'$ 时使用同一采样例程。

舍入与除法。 与通常做法一样，模二幂 r 的整数表示在区间 $[-r/2, r/2 - 1]$ 中。对于任意二幂因子 $m \mid r$ 和任意 $x \in [-r/2, r/2 - 1]$ ，均可唯一写成 $x = x_0 + x_1 m \bmod r$ ，其中 $x_0 \in [-m/2, m/2 - 1]$ and $x_1 \in [-r/(2m), r/(2m) - 1]$ 。可按 $x_0 = x \bmod m$ 、 $x_1 = (x - x_0)/m \bmod r/m$ 计算。 $\mathbf{b} = \lfloor \frac{\alpha}{\beta} \mathbf{t} \rfloor$ 返回舍入后的环元素，对应于 $\mathbf{b} = (\mathbf{t} + \frac{\beta}{2\alpha}) \gg (\log_2 \beta - \log_2 \alpha)$ ，其中 $\gg$ 表示右移，所有运算均逐系数执行。

我们用常数因子平移舍入误差，使其均值为零并改进安全性证明。更多细节参见 D’Anvers 的博士论文 [D’V28, 第 3.2 节]。

密文压缩。 为缩小密文，应用有损函数 $\lfloor \frac{t}{p} \cdot \rfloor$ ，其中 t 随参数集而定。该操作在缩小密文的同时会提高失败概率；详细分析见第 5.2.1 节。公钥不作压缩。

密钥、密文与消息的编码。 密钥、密文和消息序列化为小端序紧凑字节数组。描述中以有符号整数表示元素，但序列化时按无符号整数打包和解包。字节串存储无符号代表元，解包时再映射回中心化有符号代表元，从而避免符号扩展的实现问题。对于 b 比特系数序列 $(t_i)_{0 \leq i < N}$ ，定义

$$\text{Enc}_b(a) = (2^{b-1} - 1 - a) \bmod 2^b \in \{0, \dots, 2^b - 1\}$$

and

$$\text{Pack}((t_i)_{0 \leq i < N}, b) = \text{Enc}_b(t_0) \parallel \dots \parallel \text{Enc}_b(t_{N-1})$$

为其序列化；串接约定使 t_0 存于最低有效位置，且 $\text{Enc}_b(t_0) \bmod 2$ 对应比特 0。解包应用逆映射 $\text{Dec}_b(u) = 2^{b-1} - 1 - u$ ，返回 $[-2^{b-1}, 2^{b-1} - 1]$ 中的系数。对于系数为 b 比特的环元素 $\mathbf{t} = \sum_{i=0}^{k-1} (\sum_{j=0}^{n-1} t_{i,j} y^j) x^i \in \mathbf{S}_{q,n,k}$ ，定义其序列化为

$$\text{Pack}(\mathbf{t}, b) = \text{Pack}((t_{0,j})_{0 \leq j < n}, b) \parallel \dots \parallel \text{Pack}((t_{k-1,j})_{0 \leq j < n}, b),$$

同样将 $\text{Pack}((t_{0,j})_{0 \leq j < n}, b)$ 存于最低有效地址。

- PKE 公钥 pk_{pke} 的序列化定义为

$$\text{pkEncode}(\text{seed}_A, t) = \text{seed}_A \parallel \text{Pack}(t, \epsilon_p).$$

相应的反序列化函数为 $\text{pkDecode}(pk_{\text{pke}})$ 。PKE 公钥总大小为 $64 + \epsilon_p kn/8$ 字节。

- PKE 私钥 sk_{pke} 的序列化定义为

$$\text{skEncode}(s) = \text{Pack}(s, \log_2(2\eta)).$$

相应的反序列化函数为 $\text{skDecode}(sk_{\text{pke}})$ 。PKE 私钥总大小为 $\log_2(2\eta)kn/8$ 字节。

- PKE 密文 ct_{pke} 的序列化定义为

$$\text{ctEncode}(c_m, \mathbf{b}') = \text{Pack}((\zeta_{0,i})_{0 \leq i < n}, \epsilon_t) \parallel \cdots \parallel \text{Pack}((\zeta_{\ell-1,i})_{0 \leq i < n}, \epsilon_t) \parallel \text{Pack}(\mathbf{b}', \epsilon_p),$$

其中 $c_m = (\zeta_0, \dots, \zeta_{\ell-1}) \in \mathbf{R}_{t,n}^\ell$ 。相应的反序列化函数为 $\text{ctDecode}(ct_{\text{pke}})$ 。PKE 密文总大小为 $\epsilon_t \ell n/8 + \epsilon_p kn/8$ 字节。

- PKE 消息 m 的序列化定义为

$$\text{mEncode}(m) = \text{Pack}((\mu_{0,i})_{0 \leq i < n}, 1) \parallel \cdots \parallel \text{Pack}((\mu_{\ell-1,i})_{0 \leq i < n}, 1),$$

其中 $m = (\mu_0, \dots, \mu_{\ell-1}) \in \mathbf{R}_{2,n}^\ell$ 。因 $b = 1$ ，解码返回 $\{-1, 0\}$ 中的中心化代表元，并将其模 2 解释为 $\mathbf{R}_{2,n}$ 的元素。相应反序列化函数为 $\text{mDecode}(m)$ 。PKE 消息总大小为 $\ell n/8$ 字节。

- KEM 公钥的序列化与 PKE 公钥的序列化相同。
- KEM 私钥 sk_{KEM} 的序列化定义为 $sk_{\text{pke}} \parallel pk_{\text{pke}} \parallel hpk \parallel z$ 。
- KEM 密文的序列化与 PKE 密文相同： $ct_{\text{KEM}} = ct_{\text{pke}}$ 。
- KEM 共享秘密单独序列化为 ss ；它是 Encaps 的输出，不作为 KEM 密文的一部分传输。

Fujisaki–Okamoto 变换。 本构造将带有隐式拒绝的 Hofheinz–Hövelmanns–Kiltz 模块化 Fujisaki–Okamoto 变换 U_m^ℓ [HHK17] 应用于 Mithril.PKE 的确定性重新加密形式。在该变换中，封装阶段导出候选共享秘密时不包含密文。ROM 证明及后续 QROM 讨论见第 5.1.2 节。

5 安全性声明与分析

5.1 理论安全性

Mithril 的主要底层困难问题是环 $\mathbf{R}_q$ 上的 Ring-LWR，定义 2 对其作了回顾。对于 Mithril (以及 Octarine)， $\mathbf{R}_q$ 实例化为根式环 $\mathbf{S}_{q,n,k}$ ，该实例称为 RR-LWR 问题。安全性证明大体按如

下步骤进行：

1. 将 PKE 方案的安全性归约到基于 Ring-LWR 的类似 PKE 设计。具体是证明 $\text{Mithril} := \text{Mithril}_\ell$ 的安全性可归约到 Mithril_k ，且前者失败概率不大于后者；其中 k 为根式环扩张次数。见定理 3。
2. 应用标准安全性证明，将 Mithril_k 的安全性归约到 Ring-LWR 问题，并用标准变换得到 IND-CCA 安全的 KEM。见第 5.1.1 节和第 5.1.2 节。
3. 最后，在第 5.1.3 节证明根式环选择 $\mathbf{R}_q := \mathbf{S}_{q,n,k}$ 是安全的 Ring-LWR 实例。

Definition 2 (判定 Ring-LWR). 设 $q > p \geq 2$ ， $\mathbf{R}_\nu := \mathbf{Z}_\nu[x]/(f(x))$ 为 $\nu \in \{p, q\}$ 时的环， D_s 为 $\mathbf{R}_q$ 上的秘密分布。对从 D_s 采样的固定秘密 $\mathbf{s} \in \mathbf{R}_q$ ，判定 Ring-LWR 问题要求区分如下两个分布：

Real : $(\mathbf{A}_i, \lfloor \frac{p}{q} \mathbf{A}_i \mathbf{s} \rfloor)$ ，其中 $\mathbf{A}_i \leftarrow \mathbf{R}_q$ 在 $i = 1, \dots, N_{\text{samp}}$ 时均匀随机；

Rand : $(\mathbf{A}_i, \mathbf{b}_i)$ ，其中 $\mathbf{A}_i \leftarrow \mathbf{R}_q, \mathbf{b}_i \leftarrow \mathbf{R}_p$ 在 $i = 1, \dots, N_{\text{samp}}$ 时均匀随机。

区分器 $\mathcal{A}$ 的优势定义为

$$\text{Adv}_{q,p,D_s,N_{\text{samp}}}^{\text{R-LWR}}(\mathcal{A}) = |\Pr[\mathcal{A}(\text{Real}) = 1] - \Pr[\mathcal{A}(\text{Rand}) = 1]|.$$

判定 Ring-LWR 假设断言，对所有高效区分器 $\mathcal{A}$ ，该优势均可忽略。

5.1.1 Mithril.PKE

按标准方式定义 IND-CPA 实验。敌手收到公钥 pk ，输出两个等长消息 $m_0, m_1 \in \mathbf{R}_{2,n}^\ell$ ，随后收到隐藏比特 b 所选消息 m_b 的密文；若猜中 b 则获胜。其优势记为

$$\text{Adv}_{\text{Mithril}_\ell.\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

首先证明 Mithril_ℓ 的安全性可归约到 Mithril_k 的安全性。

Theorem 3. 对任意参与 Mithril_ℓ IND-CPA 实验的敌手 $\mathcal{A}$ ，均存在参与 Mithril_k IND-CPA 实验的敌手 $\mathcal{B}$ ，使得

$$\text{Adv}_{\text{Mithril}_\ell.\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \text{Adv}_{\text{Mithril}_k.\text{PKE}}^{\text{IND-CPA}}(\mathcal{B}).$$

此外，若以 δ_ℓ 表示 Mithril_ℓ 的失败概率，则 $\delta_\ell \leq \delta_k$ 。

证明. 设 $pk = (\text{seed}_A, \mathbf{b})$ 为 $\mathcal{B}$ 的挑战者生成的公钥。 $\mathcal{B}$ 将其转发给 $\mathcal{A}$ 。 $\mathcal{A}$ 返回两个 ℓn 比特挑战消息 m_0, m_1 。 $\mathcal{B}$ 将二者嵌入较大 kn 比特消息的最高 ℓn 比特，得到 $\mu_0 = 0^{(k-\ell)n} \| m_0$ 和 $\mu_1 = 0^{(k-\ell)n} \| m_1$ ，并发送给自己的挑战者。挑战者返回密文 $c = (c_\mu, \mathbf{b}')$ ， $\mathcal{B}$ 将其转换为 $c^* = (c_\mu^{[k-\ell:k-1]}, \mathbf{b}')$ ，这是 Mithril_ℓ 的有效密文。最后， $\mathcal{A}$ 收到 c^* 后返回比特 b' ， $\mathcal{B}$ 将其回复给挑战者。由于 c^* 对应 Mithril_ℓ 对 m_b 的加密，而 c 对应 Mithril_k 对 μ_b 的加密， $\mathcal{A}$ 与 $\mathcal{B}$ 的优势完全相同。

按 PKE 设计有 $\ell \leq k$, 故 Mithril_ℓ 的密文是 Mithril_k 密文的一部分。因此, 若 Mithril_ℓ 解密失败, Mithril_k 也会失败, 于是 $\delta_\ell \leq \delta_k$ 。□

由于 Mithril_k 是常规 Ring-LWR PKE, 可应用到 Ring-LWR 问题的标准归约。D’Anvers 的论文 [DVV28, 定理 2–3] 总结了 Saber 背景下 Ring-LWR 类方案的安全性证明。D’Anvers 处理的是 Module-LWR, 但令 $l = 1$ 即可恢复 Ring-LWR。由该结果可知, 在 ROM 和 $q/p \leq p/T$ 条件下, Mithril 的 IND-CPA 安全性可归约到 $\text{Adv}_{q,p,\mathcal{M}_2}^{\text{R-LWR}}$ 。与 Saber 相同, 我们以此作为参数选择依据。

5.1.2 Mithril.KEM

对 Mithril_ℓ 使用标准 IND-CCA KEM 实验。本构造遵循 Hofheinz–Hövelmanns–Kiltz [HHK17] 的带隐式拒绝模块化 FO 变换 U_m^χ , 并将其应用于 Mithril.PKE 的确定性重新加密形式。Hofheinz–Hövelmanns–Kiltz 首先建立其 QROM 安全性; Bindel et al. [BHH⁺19] 以及 Don–Fehr–Majenz–Schaffner [DFMS19, DFMS21] 通过“测量并重编程”框架加以改进; Kuchta et al. [KSS⁺20] 又使用“测量–回绕–测量”进一步收紧界限。

将这些结果应用于本 PKE, 在量子查询界 q_G 和 q_H 下可得到如下形式的界:

$$\text{Adv}_{\text{Mithril.KEM}}^{\text{IND-CCA,QROM}}(\mathcal{A}) \leq C \cdot q_G \cdot \sqrt{\text{Adv}_{\text{Mithril.PKE}}^{\text{OW-CPA,QROM}}(\mathcal{B})} + \mathcal{O}\left(\frac{q_G^2 + q_H^2}{2^{\ell n}}\right) + q_{\text{dec}} \delta_{\text{dec}} + \Delta_{\text{coll}}^q,$$

其中 C 为绝对常数; 文献 [HHK17] 可直接给出不含平方根、但带乘法因子 q_G^2 的较粗界。

5.1.3 RR-LWR 实例的安全性

设 n 为 2 的幂, k 为严格正整数。可以证明多项式 $f(x) = (x^k - 2)^n + 1$ 不可约 [BRVW26, 引理 1], 因此 $\mathbf{Q}[x]/(f(x))$ 是数域, 可在其上定义 Ring-LWR 问题。这是标准 LWR [BPR12] 的数域对应物, 并以 Mithril 使用的根式环表示实例化。

由多项式 $f(x) = (x^k - 2)^n + 1$, 在数域 $\mathbf{K} = \mathbf{Q}[x]/(f(x)) = \mathbf{Q}[\alpha]$ 的整数环 $\mathcal{O}_{\mathbf{K}}$ 上定义 RR-LWR 问题。与 Ring-LWE 不同, LWR 样本不含独立采样的 Gaussian 误差项; 其噪声是下述逐系数映射引入的确定性量化误差:

$$z \in \mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}} \mapsto \left\lfloor \frac{p}{q}z \right\rfloor \in \mathcal{O}_{\mathbf{K}}/p\mathcal{O}_{\mathbf{K}}.$$

通过显式计算 (例如使用 Dedekind 判据) 可验证 $\mathcal{O}_{\mathbf{K}} = \mathbf{Z}[\alpha]$ 。因此, 环

$$S_{q,n,k} = (\mathbf{Z}_q[y]/(y^n + 1)[x]) / (x^k - y - 2)$$

只是 $\mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}}$ 的具体系数表示。下述定理正式陈述该表示归约。

Theorem 4 (根式数域上的基系数 Ring-LWR 形式 RR-LWR). 设 $q > p \geq 2$ 且 $p \mid q$, $f(x) = (x^k - 2)^n + 1$ 不可约, $\mathbf{K} = \mathbf{Q}[x]/(f(x)) = \mathbf{Q}[\alpha]$ 且 $\mathcal{O}_{\mathbf{K}} = \mathbf{Z}[\alpha]$ 。令

$$\varphi_q : S_{q,n,k} \longrightarrow \mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}}$$

为由 $x \mapsto \alpha$ 、 $y \mapsto \alpha^k - 2$ 诱导的环同构，并类似地定义模 p 的 φ_p 。令 $\mathcal{B} = \{(\alpha^k - 2)^i \alpha^j : 0 \leq i < n, 0 \leq j < k\}$ 为 $\mathcal{O}_{\mathbf{K}}$ 的 $\mathbf{Z}$ 基，它是单项式基 $\{y^i x^j\}$ 在 φ_q 下的像；令 $\lfloor \cdot \rfloor_{\mathcal{B}}$ 表示在基 $\mathcal{B}$ 中逐系数执行、从 $\mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}}$ 到 $\mathcal{O}_{\mathbf{K}}/p\mathcal{O}_{\mathbf{K}}$ 的舍入。对于 $S_{q,n,k}$ 上任意秘密分布 D_s ，任何区分如下 RR-LWR 样本

$$\left(\mathbf{A}, \left\lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \right\rfloor \right) \in S_{q,n,k} \times S_{p,n,k}, \quad \mathbf{A} \leftarrow S_{q,n,k}, \quad \mathbf{s} \leftarrow D_s,$$

与均匀样本的区分器 $\mathcal{A}$ ，均可导出 $\mathcal{O}_{\mathbf{K}}$ 上相应判定 Ring-LWR 问题的区分器 $\mathcal{B}'$ 。后者使用舍入 $\lfloor \cdot \rfloor_{\mathcal{B}}$ 和秘密分布 $\varphi_q(D_s)$ ，样本数相同、运行时间基本相同、优势也相同。

证明. 由于 $S_{q,n,k} \cong \mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}}$ 且 $S_{p,n,k} \cong \mathcal{O}_{\mathbf{K}}/p\mathcal{O}_{\mathbf{K}}$ (后者使用 $\mathcal{O}_{\mathbf{K}} = \mathbf{Z}[\alpha]$)，映射 φ_q 和 φ_p 均为可高效计算的双射。RR-LWR 使用的舍入在基 $\{y^i x^j\}$ 中逐系数执行，而 φ_q 将该基映到 $\mathcal{B}$ 。因此按 $\lfloor \cdot \rfloor_{\mathcal{B}}$ 的定义，对每个 $z \in S_{q,n,k}$ ，有

$$\varphi_p \left(\left\lfloor \frac{p}{q} z \right\rfloor \right) = \left\lfloor \frac{p}{q} \varphi_q(z) \right\rfloor_{\mathcal{B}}.$$

归约算法 $\mathcal{B}'$ 收到 Ring-LWR 挑战样本 $(\tilde{\mathbf{A}}_i, \tilde{\mathbf{b}}_i) \in (\mathcal{O}_{\mathbf{K}}/q\mathcal{O}_{\mathbf{K}}) \times (\mathcal{O}_{\mathbf{K}}/p\mathcal{O}_{\mathbf{K}})$ ，并将其映射为

$$(\mathbf{A}_i, \mathbf{b}_i) = (\varphi_q^{-1}(\tilde{\mathbf{A}}_i), \varphi_p^{-1}(\tilde{\mathbf{b}}_i)).$$

若挑战样本是秘密 $\tilde{\mathbf{s}} = \varphi_q(\mathbf{s})$ 下的 Ring-LWR 样本，则映射所得恰为秘密 $\mathbf{s}$ 下的 RR-LWR 样本；若挑战均匀，则因 φ_q, φ_p 为双射，映射后的样本仍均匀。因此 $\mathcal{B}'$ 与 $\mathcal{A}$ 具有相同优势。 $\square$

需要强调，定理 4 将 RR-LWR 归约到一种 Ring-LWR 变体。该变体在传输后的单项式基 $\mathcal{B}$ 中逐系数舍入，而非在典范嵌入中舍入。它与 Ring-LWR 典范嵌入版本的关系由下述线性换基 L 描述：其奇异值 $\sigma_{\min}(L) = \sqrt{nk}$ 、 $\sigma_{\max}(L) \leq \beta \sqrt{nk}$ (小常数 β 见下文) 说明，在任一基中的困难性都可转移，秘密/误差范数最多增加因子 β ；该因子被纳入具体攻击估计。

由 f 可显式写出其 nk 个根：

$$r_{i,j} = (\xi_{2n}^{(2j+1)} + 2)^{(1/k)} \xi_k^i$$

其中 $i = 0, \dots, k-1$ 、 $j = 0, \dots, n-1$ ， ξ_m 为本原 m 次单位根。令这些根按 $r_{m+nk/2} = \overline{r_m}$ ($m = 1, \dots, nk/2$) 排序，则元素 $a(x) \in \mathbf{K}$ 的典范嵌入由 nk 个求值 $a(r_i)$ ($i = 1, \dots, nk$) 组成。

为比较本 RR-LWR 实例与通常的典范嵌入视角，还需考察在环表示 $(\mathbf{Z}_q[y]/(y^n + 1)[x])/(x^k - y - 2)$ 的基 $y^i x^j$ ($i = 0, \dots, n-1$ ， $j = 0, \dots, k-1$) 中逐系数采样，与 $\mathbf{K}$ 的典范嵌入之间的差异。该差异是由两个线性变换复合而成的线性变换 L ：

- 第一个变换将基 $y^i x^j$ 映射到对应元素 $b(x) = (x^k - 2)^i x^j \bmod f(x)$ (回忆本环中确有 $y = x^k - 2$)；
- 第二个变换是标准典范嵌入，将 $b(x)$ 映射到 $[b(r_i)]_{i=1, \dots, nk}$ 。该变换即 Vandermonde 矩阵，其第 i 行由 r_i^j ($j = 0, \dots, nk-1$) 的幂组成。

要约束在基 $y^i x^j$ 中采样相对于典范基采样引入的最大畸变，只需考察 L 的最小和最大奇异值。在二幂分圆多项式 $x^n + 1$ 的“理想情形”中，二者相同，均为 $\sqrt{n}$ 。我们对 $k = 2, \dots, 15$ 及

方案	类型	C	Q	β	精化攻击	失败概率
Mithril-128	primal	134	121	458	153	-603
	dual-hybrid	129	117	443		
Mithril-256	primal	270	245	925	277	-280
	dual-hybrid	257	233	879		
Mithril-512	primal	559	507	1913	539	-230
	dual-hybrid	526	478	1802		
ML-KEM-512	primal	119	108	406	140	-139
	dual-hybrid	115	105	395		
ML-KEM-768	primal	182	165	624	196	-165
	dual-hybrid	174	158	597		
ML-KEM-1024	primal	255	232	874	262	-175
	dual-hybrid	241	219	826		
LightSaber	primal	118	107	405	140	-120
	dual-hybrid	122	111	419		
Saber	primal	190	172	649	205	-136
	dual-hybrid	182	165	624		
FireSaber	primal	260	237	891	267	-165
	dual-hybrid	248	225	848		

表 3: Mithril 与相关格基方案的安全性估计、精化攻击成本及失败概率比较。

不同 n 计算了 L 的奇异值, 发现最小奇异值为 $\sqrt{nk}$ (nk 为扩张次数, 故这是最优的), 最大奇异值为依赖 k 的小倍数 $\beta\sqrt{nk}$: $k=2$ 时 $\beta < 1.74$, $k=3$ 时 $\beta < 2.08$, $k=5$ 时 $\beta < 2.41$, $k=7$ 时 $\beta < 2.57$, $k=15$ 时 $\beta < 2.79$ 。因此, 在系数基中采样相对典范基采样只产生很小畸变。由此, 可将 Mithril 使用的秘密分布解释为次数 kn 数域上的标准 Ring-LWR 秘密分布, 只带上述有界换基畸变; 进而可用维数 kn 的一般格攻击估计所得 RR-LWR 实例的困难性。

最后, 由于 $f(x)$ 不可约, 本环结构不同于 m-RLWE 文献 [PUTPPG15] 及后续工作 [PUTPPG16, APUPG17, PUTPG+21] 所考虑的多元环。那些工作考察形如 $R_q[x, y] = \mathbf{Z}_q[x, y]/(g(x), h(y))$ 的环, 其中 g, h 为定义在 $\mathbf{Z}$ 上的一元多项式。Bootland、Castryck 与 Vercauteren [BCV20] 破解了某些 g, h 选择下的 m-RLWE, 尤其是二者的根均位于扩张次数小于 $\deg(g)\deg(h)$ 的数域中时。式 (5) 的环模多项式 $f(x) = (x^k - 2)^n + 1$ 确实定义次数 kn 的数域, 故这些攻击不适用。虽然我们利用式 (5) 的同构高效使用环结构, 底层格问题只是一元环 $S_{q,n,k}$ 上的 Ring-LWR 问题。

5.1.4 具体攻击估计方法

评估 Mithril 安全性时, 首先采用广泛使用的 Core-SVP 困难性方法 [ADPS16]。该方法与具体方案无关, 可比较不同格基密码系统的参数强度。Core-SVP 指标实质上衡量在所选参数下求解底层格问题 (例如由 RR-LWR 导出的 unique-SVP 实例) 的困难程度, 并假定最佳攻击采用格约化 (如 BKZ [SE94, CN11]) 及块维数 β 下的最优 SVP 预言机。忽略格约化的多项式时间开销和内存成本等低阶因素后, 攻击成本估计为关于 BKZ 块大小 β 的指数函数。假定调用一次理想长度 β 的 SVP 求解器, 在经典环境下约需 $2^{0.292\beta}$ 次基本运算; 量子攻击者约需 $2^{0.265\beta}$ 次运算, 其中考虑了筛法 SVP 算法已知最佳的 Grover 型加速 [Laa15, LMvdP15]。该粗粒度

指标主要依赖格维数和噪声率，而非方案的具体代数结构，便于直接比较不同格基提案；同时 Core-SVP 往往较保守，有利于选择即使在敌手成本模型较乐观时仍可抵抗攻击的参数。

对于可进一步归约到 LWE 的 RR-LWR 类方案，两类主要格攻击为原始攻击 (primal attack) 和对偶混合攻击 (dual-hybrid attack)。原始攻击将 LWE 实例归约为 uSVP 实例，并尝试通过格约化直接恢复秘密；对偶混合攻击则将对偶格区分器与对部分秘密的组合猜测步骤结合。在 Core-SVP 框架中，通常确定使相应区分或解码步骤成功所需的 BKZ 块大小 β ，再换算为估计成本。

Core-SVP 提供简洁稳健的基线，但有意抽象掉实际攻击复杂度的许多方面。因此，我们还使用 Albrecht et al. 的 lattice-estimator 框架 [APS15] 给出精化成本评估。其中 `estimate.rough` 主要从 BKZ 块大小 β 导出安全等级，与 Core-SVP 方法紧密对应，并提供一致、与方案无关的参考点。

为获得更现实的估计，我们还使用完整 `estimate` 函数，显式计入完整攻击流程所需环运算数 (rop) 以精化攻击成本。除底层 SVP 预言机的渐近成本外，还包括 BKZ 约化开销、free-dimension 技术、混合策略及其他算法优化。因此，精化估计覆盖 Core-SVP 忽略的因素，如重复调用 SVP 的成本、不可忽略的多项式因子以及时间-内存实践权衡。

从概念上看，精化估计可更准确反映具体安全性，尤其是经典攻击；近期格约化和筛法进展已显著提升其实际性能。但贴近现实运行也会引入更多假设：结果依赖于 BKZ 的行为、筛法实现有效性及底层运算成本模型有效性的启发式判断。因此，精化估计容易随时间变化而变化，也更易受未来算法改进影响。

我们因此同时报告两种指标：Core-SVP 困难性作为保守、可比的基线；基于 rop 的精化估计反映当前对具体攻击成本的最佳认识。这种双重视角符合格基方案分析的标准做法，即以更详细但依赖假设的评估补充粗粒度而稳健的指标。

表 2 列出 Mithril 各参数集的安全等级。表 3 将其与基于 LWE 的 ML-KEM 及基于 LWR 的 Saber 比较。数值计算细节见附录 A。

5.2 实现安全性

5.2.1 解密失败攻击

与多数 LWE/LWR 类方案一样，解密并非始终完全正确。当确定性舍入误差与私钥、密文向量之间的相互作用累积到足以使解密系数越过解码边界时，即发生解密失败。此类错误可被检测，并可能泄漏私钥信息。

解密失败攻击主动利用这种泄漏。敌手有意构造特定密文以人为放大误差项 [DGJ⁺19, DRV20, DB22]。通过观察解密成功或失败，攻击者获知关于私钥系数的精确线性不等式；系统地积累足够多的诱发失败密文后，可构造不等式系统，并用统计技术或格约化算法求解，从而完全恢复私钥。

Mithril 的解密失败与 Saber 等其他 Ring-LWR 或 Module-LWR 方案类似。已有工作分析了解密失败的计算方法，例如文献 [DKR⁺20, 定理 6.2]。按照本规范的记号，解密失败概率可计算为

$$\delta = \Pr[\|(\mathbf{b}\mathbf{s}')^{[k-\ell:k-1]} - (\mathbf{b}'\mathbf{s})^{[k-\ell:k-1]} + \mathbf{e}_r, \text{ mod } p\|_\infty > p/4]$$

其中 $\mathbf{e}_r \in \mathbf{R}_q^\ell$ 为密文舍入误差, 系数位于 $[-p/(2t), p/(2t) - 1]$ 。我们与先前工作一样, 假定各共享秘密比特的失败概率可独立建模。

由于本方案在根式环中运算, 需要注意 $(\mathbf{b}\mathbf{s}')^{[k-\ell:k-1]}$ and $(\mathbf{b}'\mathbf{s})^{[k-\ell:k-1]}$ 均为 $\mathbf{R}_q^\ell$ 的元素, 而 Module-LWR 通常使用 $\mathbf{R}_q$ 的元素。换言之, 私钥比特从 ℓ 个各为次数 n 的多项式系数中导出, Module-LWR 则来自一个含 n 个系数的多项式。本方案由于选择 $n = 128$ 而非 $n = 256$, 在 $\ell = 1$ 的 128 比特安全等级下相较其他方案失败概率更低。

不过, 根式环结构在 $\mathbf{b}$ 与 $\mathbf{s}'$ 的第 i 行内积中包含 $k - 1 - i$ 次乘以 $y + 2$ 的运算, 见式 (6)。这会增大系数范数, 因此在失败概率计算中纳入最坏情形因子 3。计算底部行可尽量减小乘以 $y + 2$ 的影响。例如, Mithril-128 中 $\ell = 1$ 时只使用一行; 底行不涉及 $y + 2$, 故实际没有此类乘法, 乘法等价于 Module-LWR 的普通内积。

附录 A.2 的脚本估计表 2 和表 3 所列 ARCANE-Mithril 各参数集的解密失败概率。其对数远低于实际可发生的范围, 因此解密失败攻击对 Mithril 安全性的实际影响可忽略。

5.2.2 侧信道攻击

算法不含依赖秘密的分支, 因此为常数时间。二幂模数还可简化若干计算, 例如生成随机元素时避免非常数时间的拒绝采样。因此, 只要底层硬件指令也是常数时间, 正确的 ARCANE-Mithril 实现应为常数时间。

不过, 敌手可能在解封装期间观察未防护 Mithril 实现的功耗或电磁辐射 [RRCB20, XPR⁺22, RCDB24], 以猜测长期私钥。因此必须采用已知侧信道攻击对策。Mithril 计算的确定性执行方式以及采用二幂模数的 LWR 基础有利于高效实现掩码防护; Saber KEM 文献中的对策 [BDK⁺21] 可用于 Mithril。

- **二幂模数简化掩码算术**掩码、模约化、舍入、压缩以及布尔到算术的掩码转换均可用移位和掩码低开销实现。
- **确定性 LWR 舍入避免显式误差采样**这减少了必须掩码的随机性, 使舍入比 LWE 型噪声加法更易防护。
- **无拒绝的常数时间采样**每个系数使用固定数量的比特生成随机元素, 避免使常数时间和掩码实现复杂化的拒绝循环。
- **简单秘密分布支持掩码乘法**较小的有符号系数集合 $\{-2, -1, 0, 1\}$ 简化秘密系数的掩码处理, 并降低受保护算术的复杂度 [ABC⁺23]。

6 性能评估

Mithril 的算术设计有意不限定多项式乘法算法。本规范定义所需根式环运算, 但不依赖负循环多项式乘法采用的特定算法或元素表示, 因此实现可按平台或优化目标选择最合适的乘法策略。为展示这种灵活性, 参考实现采用 Toom-Cook 乘法 [Too63, Coo66], AVX2 实现则在辅助素数上使用 NTT [CT65, Pol71]。

6.1 算术实现

参考实现 参考实现使用 16 比特累加器在二幂系数环上执行算术。次数 128 的负循环子环乘法由四路 Toom–Cook 乘法实现。关系 $x^k = y + 2$ 通过预计算乘法窗口纳入：根式基中的每个所需系数均由负循环子环上长度为 k 的内积得到。因此，实现既可计算完整根式环乘积，也可只计算其最高 ℓ 个次数系数。只要可能，到二幂模数的舍入和约化就与最终累加及序列化融合。

AVX2 实现 AVX2 实现使用 26 比特辅助素数

$$53\,501\,952 + 1 = 2^{13} \cdot 3 \cdot 7 \cdot 311 + 1$$

用于长度 128 的 NTT。系数表示在 Montgomery 域中；公开操作数仅变换一次，并为该操作所需的全部根式基内积做好准备。对于仅通过承载消息的根式系数使用的公钥和密文分量，准备工作只覆盖所需的 ℓ 个输出系数。正变换、逐点乘法、累加、逆变换以及最终二幂舍入或约化均以 AVX2 向量化。该辅助素数仅为特定实现的选择：其大小足以容纳未约化的中间乘积，不改变任何 Mithril 参数或分布。

6.2 性能分析

采样与扩展 三个参数集均采用 $n = 128$ 、 $\epsilon_q = 13$ 和 2 比特秘密系数；区别在于根式扩张次数 $k \in \{5, 9, 17\}$ 、消息维数 $\ell \in \{1, 2, 4\}$ 及密文压缩模数 t 。基于种子的采样器产生比特紧凑的 XOF 输出，可直接解包，无需拒绝。扩展 **A** 需要

$$\frac{\epsilon_q n k}{8} = 208k$$

字节 XOF 输出，即 Mithril-128、Mithril-256 和 Mithril-512 分别需要 1040、1872 和 3536 字节。采样一个秘密环元素需要

$$\frac{2nk}{8} = 32k$$

字节，即分别为 160、288 和 544 字节。因此，采样成本主要由 XOF 吞吐量、比特解包以及 AVX2 实现中的变换域表示准备决定。

根式环乘法 对于完整乘积， k 个输出系数均为长度 k 的内积，因此需要

$$N_{\text{full}} = k^2$$

个基础负循环多项式乘法。只计算承载消息的 ℓ 个系数需要

$$N_{\text{sel}} = k\ell$$

个基础乘法。因此，Mithril-128、Mithril-256 和 Mithril-512 的完整乘积分别使用 25、81 和 289 个基础乘法；共享秘密计算中的多项式乘法只需计算部分输出（见第 4 节“消息空间”段），分别使用 5、18 和 68 个乘法。负循环子环次数保持不变，因此这些数量解释了参数集间的主要扩

展趋势。

运算开销 Mithril.PKE 密钥生成扩展 $\mathbf{A}$ 、采样 $\mathbf{s}$ 、计算完整乘积 $\mathbf{As}$ 、从 q 舍入到 p ，并序列化密钥对。加密扩展 $\mathbf{A}$ 、采样 $\mathbf{s}'$ 、计算完整乘积 $\mathbf{As}'$ ，随后仅计算 $\mathbf{bs}'$ 的最高 ℓ 个次数系数，再执行消息加法和密文压缩。解密不扩展 $\mathbf{A}$ ，只需计算 $\mathbf{b's}$ 中对应的 ℓ 个系数，随后恢复消息。

KEM 操作还包含第 4 节规定的 Fujisaki–Okamoto 哈希与随机性生成。解封装执行 PKE 解密，导出候选密钥和加密种子，确定性地重新加密恢复出的消息，从密文和 z 导出后备密钥，并以常数时间选择输出。解封装同时包含 PKE 解密和确定性重新加密，因此在最小参数集下其开销与其他 KEM 操作相近，在两个较大参数集下则成为开销最大的 KEM 操作。

通信与存储开销见表 2。KEM 公钥即 PKE 公钥，KEM 密文即 PKE 密文；KEM 私钥包含 PKE 私钥、PKE 公钥、公钥摘要和后备秘密 z 。

6.3 性能测试

实验设置 所有测量均在 2.30 GHz Intel(R) Xeon(R) CPU E5-2686 v4 上完成，该平台支持 AVX2、BMI2 和 POPCNT，运行 Linux 7.0.0-1006-aws，编译器为 GCC 15.2.0。参考实现以 C 编写，AVX2 实现结合 C 与 x86-64 汇编。参考实现使用以下选项编译：-O3、-march=x86-64-v3、-mtune=sapphirerapids、-mprefer-vector-width=256、-fomit-frame-pointer、-fno-stack-protector，and -DNDEBUG。AVX2 实现使用 -O3、-fomit-frame-pointer、-mavx2、-mbmi2、-mpopcnt、-march=native，and -mtune=native。编译。表 4 和表 5 所列基于 pseudoXOF 的提交测量中，基准二进制文件使用提交包内基于 pseudoXOF 的构造实例化 XOF 与哈希函数。

每个基准二进制文件内部重复执行 1000 次，并使用 taskset -c 0 在 CPU 0 上串行运行。计时采用 x86 时间戳计数器，并扣除测得的时间戳开销。各表给出 RDTSC tick 的中位数，数值越低越快。完整函数测量包含相应算法调用执行的采样、哈希、算术、打包及其他全部工作。Xen 客户机未暴露 CPU 频率调节器或睿频控制，因此这些设置保持 VM/提供商默认值。下文 ML-KEM 比较使用本地源码树中 Git 修订版 4768bd37c02f 的实现。

端到端性能 表 4 和表 5 给出提交包内基于 pseudoXOF 的参考实现与 AVX2 实现的端到端 PKE、KEM 操作开销。

参数集	实现	KeyGen	Enc	Dec
Mithril-128	参考实现	180 771	194 212	13 647
	优化实现	127 580	130 703	5 600
Mithril-256	参考实现	407 536	452 273	45 384
	优化实现	230 001	236 790	10 651
Mithril-512	参考实现	1 100 406	1 266 727	163 574
	优化实现	439 142	452 982	21 550

表 4: 参考实现与优化实现中基于 pseudoXOF 的 Mithril.PKE 端到端性能。数值为 RDTSC tick 的中位数。

参数集	实现	KeyGen	Encaps	Decaps
Mithril-128	参考实现	219 236	224 906	234 140
	优化实现	166 169	161 615	162 914
Mithril-256	参考实现	462 721	502 753	546 039
	优化实现	284 311	287 839	295 922
Mithril-512	参考实现	1 256 827	1 430 214	1 612 251
	优化实现	595 321	619 692	656 092

表 5: 参考实现与优化实现中基于 `pseudoXOF` 的 Mithril.KEM 端到端性能。数值为 RDTSC tick 的中位数。

AVX2 加速比随 k 增大, 但端到端收益小于纯算术收益, 因为基于 `pseudoXOF` 的采样和哈希仍占测量路径的较大部分。三个参数集的 PKE 密钥生成加速比分别为 $1.42\times$ 、 $1.77\times$ 、 $2.51\times$, PKE 加密分别为 $1.49\times$ 、 $1.91\times$ 、 $2.80\times$ 。PKE 解密分别加速 $2.44\times$ 、 $4.26\times$ 、 $7.59\times$, 因为算术占比更高且只计算选定的 ℓ 个系数。KEM 加速比为 $1.32\times$ 至 $2.46\times$; 较小的加速比源于密钥生成、封装和解封装中额外的 `pseudoXOF` 哈希。

算术开销分解 表 6 单独列出参考实现中的主要算术开销。

参数集	基础 Toom–Cook	完整乘积	单个选定系数	PKE Dec
Mithril-128	2 313	57 737	11 537	13 647
Mithril-256	2 313	186 540	20 731	45 384
Mithril-512	2 304	669 901	39 119	163 574

表 6: 参考实现中选定算术操作的开销。“单个选定系数”列测量一个最高次数根式系数; PKE Dec 包含全部 ℓ 个所需系数及消息恢复。数值为 RDTSC tick 的中位数。

由于 n 固定, 次数 128 的 Toom–Cook 乘法开销基本不变。完整乘积测量紧随基础乘积数量 k^2 增长, 单系数测量则随 k 线性增长。端到端解密路径随 $k\ell$ 增长, 另包括解包、约化、舍入和消息序列化。

表 7 给出 AVX2 实现的相应微基准结果。

参数集	扩展并准备 A	采样秘密	变换秘密	完整乘积与舍入	ℓ 系数乘积与约化
Mithril-128	105 819	14 415	2 000	5 569	2 751
Mithril-256	190 339	25 840	3 595	11 053	5 351
Mithril-512	359 853	48 752	6 793	25 033	10 982

表 7: AVX2 实现中选定采样与算术操作的开销。两个乘积列均为端到端内核, 已包含秘密变换, 故各列不可相加。数值为 RDTSC tick 的中位数。

扩展并准备 **A** 及变换秘密的开销主要随 k 增长; 完整乘积内核则反映变换域准备后根式基结构的 k^2 增长。选定系数乘积只计算 ℓ 个输出系数, 增长较慢。每个操作数的变换域表示只准备一次, 并在同一操作的所有内积间复用; 结果不假定在独立调用之间进行缓存。

与 ML-KEM 的比较 表 8 在同一平台上比较附加实现目录中基于 SHAKE 的 AVX2 Mithril 实现与 ML-KEM-512 AVX2、ML-KEM-1024 AVX2。Mithril-128 与 ML-KEM-512 比较，Mithril-256 与 ML-KEM-1024 比较。由于 ML-KEM 没有面向相应 512 比特经典安全等级的参数集，省略 Mithril-512。这些均为端到端比较，每项测量包含被测实现的哈希套件、序列化和随机性生成，因此差异不能仅归因于多项式算术。

操作	Mithril-128 AVX2	ML-KEM-512 AVX2	Mithril-256 AVX2	ML-KEM-1024 AVX2
PKE KeyGen	19 636	14 064	33 355	37 725
PKE Enc	22 979	15 147	40 195	39 272
PKE Dec	5 612	1 399	10 657	2 622
KEM KeyGen	35 191	24 408	56 470	56 238
KEM Encaps	35 405	26 000	60 055	57 583
KEM Decaps	39 232	27 460	69 592	62 177

表 8: 基于 SHAKE 的 AVX2 实现与 ML-KEM AVX2 的端到端比较。数值为 RDTSC tick 的中位数。

引入 ML-KEM 测量是为了以成熟且高度优化的格基 KEM 实现作为外部校准点，而非进行严格的同等安全或逐项等价比较。被比较参数集的安全裕量、编码数据大小和代数假设均不同。在第一个 AVX2 校准点，ML-KEM-512 的六项操作周期中位数均低于 Mithril-128。这与 Mithril-128 面向更强估计格安全裕量相一致：Mithril-128 的经典、量子 Core-SVP 开销估计分别为 129、117 比特，精化门数估计为 153 比特；ML-KEM-512 相应为 115、105、140 比特。

在 ML-KEM-1024 级校准点，AVX2 结果更依赖具体操作。Mithril-256 的 PKE 密钥生成快约 11.6%，KEM 密钥生成中位数相差不足 0.5%，表明该操作性能实际上相当。ML-KEM-1024 在 PKE 加密、PKE 解密、KEM 封装和 KEM 解封装中的中位数更低；但仍须结合不同安全裕量理解：Mithril-256 的经典、量子 Core-SVP 开销估计为 257、233 比特，精化估计为 277 比特；ML-KEM-1024 相应为 241、219、262 比特。

这些差异也符合两种方案不同的算术结构。ML-KEM 在固定次数 256 的分圆环上使用小模秩 Module-LWE，而 Mithril 在根式环表示上使用 RR-LWR 算术，其中 $n = 128$ 、 $k \in \{5, 9, 17\}$ ，消息提取由 ℓ 控制。AVX2 实现中的选定系数优化减少了解密和解封装所需的根式环输出量，但并未消除后端的扩展、变换与准备开销。因此，测得性能应视为在更强 RR-LWR 参数裕量下当前 AVX2 实现的实践基线，而不是与 ML-KEM 的直接等成本比较。

致谢

感谢三位博士研究生张世聪、衷琛鑫和黄浩宸参与算法设计的技术讨论，并为提交材料的准备作出贡献。他们分别为提交材料的不同部分提供了支持：张世聪参与算法描述，衷琛鑫参与安全性证明，黄浩宸参与软件实现。

附录

A SageMath 脚本

A.1 安全性估计

我们使用 Albrecht 等人 [APS15] 的 lattice-estimator 框架估计 RR-LWR 实例的具体困难性。具体使用 [malb/lattice-estimator](#), 的 lattice-estimator 实现, 提交版本为 [be81b18](#)。

为报告可与 Core-SVP 比较的安全性数值 (格密码文献中的常见做法), 我们使用估计器的“粗略”接口 `LWE.estimate.rough`; 对更精细的度量则调用 `LWE.estimate`。

在 lattice-estimator 仓库中运行下述脚本, 可复现表 2 和表 3 所列的经典与量子 Core-SVP 比特安全性数值及精细攻击估计。为得到 Mithril-512 的安全性估计, 需要在 `estimator/conf.py` 中将块大小上界调为

```
1 max_beta = 3500

1 from estimator import *
2 from estimator.nd import UniformMod
3 from estimator.lwe_parameters import LWEParameters
4
5
6 def print_core_svp_table(results):
7     schemes = list(results.keys())
8
9     rows = [("比特", [f"{results[s]['bits']:.1f}" for s in schemes]),
10            ("beta", [str(results[s]["beta"]) for s in schemes]),
11            ("经典 core-SVP", [f"{results[s]['classical_svp']:.1f}" for s
12                               in schemes]),
13            ("量子 core-SVP", [f"{results[s]['quantum_svp']:.1f}" for s in
14                               schemes]),
15            ("扩展 beta", [str(results[s]["beta_ext"]) for s in schemes]),
16            ("扩展 rop", [str(results[s]["bits_ext"]) for s in schemes]), ]
17
18     header = [""] + schemes
19     table = [header] + [[name] + vals for name, vals in rows]
20
21     col_widths = [max(len(str(cell)) for cell in col)
22                   for col in zip(*table)]
23
24     for row in table:
25         print(" ".join(str(cell).rjust(w)
26                         for cell, w in zip(row, col_widths)))
27
28 def best_attack(estimate):
29     return min(((results["beta"], N(log(results["rop"], 2)))
```

```

28         for attack, results in estimate.items()
29             if "beta" in results),
30             key=lambda x: x[1])
31
32 # 定义参数集
33 rrlwr5_params = LWEParameters(
34     n=5*128,
35     q=2**13,
36     Xs=UniformMod(4),
37     Xe=UniformMod(2**(13-11)), # p = 2^11
38     m=5*128,
39     tag="RR-LWR-5"
40 )
41 rrlwr9_params = LWEParameters(
42     n=9*128,
43     q=2**13,
44     Xs=UniformMod(4),
45     Xe=UniformMod(2**(13-11)), # p = 2^11
46     m=9*128,
47     tag="RR-LWR-9"
48 )
49 rrlwr17_params = LWEParameters(
50     n=17*128,
51     q=2**13,
52     Xs=UniformMod(4),
53     Xe=UniformMod(2**(13-11)), # p = 2^11
54     m=17*128,
55     tag="RR-LWR-17"
56 )
57
58 params = {
59     # 所提出的 RR-LWR 参数集
60     "rrlwr5": rrlwr5_params,
61     "rrlwr9": rrlwr9_params,
62     "rrlwr17": rrlwr17_params,
63     # 用于比较的 Kyber*
64     "kyber512": schemes.Kyber512,
65     "kyber768": schemes.Kyber768,
66     "kyber1024": schemes.Kyber1024,
67     # 用于比较的 *Saber
68     "lightsaber": schemes.LightSaber,
69     "saber": schemes.Saber,
70     "firesaber": schemes.FireSaber,
71 }
72

```

```

73 results = {}
74 for name, p in params.items():
75     estimate = LWE.estimate.rough(p, quiet=True, jobs=4)
76     beta, bits = best_attack(estimate)
77     results[name] = {"beta": beta, "bits": bits }
78     estimate_extended = LWE.estimate(p, quiet=True, jobs=4)
79     beta_ext, bits_ext = best_attack(estimate_extended)
80     results[name] = {"beta": beta, "bits": bits,
81                     "beta_ext": beta_ext, "bits_ext": bits_ext }
82
83 for r in results.values():
84     beta = r["beta"]
85     r["classical_svp"] = 0.292 * beta
86     r["quantum_svp"] = 0.265 * beta
87
88 print_core_svp_table(results)

```

A.2 失败概率

我们使用 SABER 实现 [KULeuven-COSIC/SABER](#), 中提交版本 [f7f39e4](#) 的失败概率脚本进行估计。考虑到根式环结构, 我们作了若干调整, 见第 5.2.1 节。运行下述脚本可复现表 2 和表 3 中报告的失败概率。

```

1  #
2  # 本文件改编自以下实现
3  # (公共领域, 可在此获取: https://github.com/KULeuven-COSIC/SABER/blob/master/Scripts/Python\_Scripts/select\_params/proba\_util.py)
4  #
5
6  from math import factorial as fac
7  from math import log, ceil, erf, sqrt, floor
8  import decimal
9
10 decimal.getcontext().prec = 1000
11
12 def mod_switch(x, q, rq):
13     """ 模数切换 (舍入到环面的另一种离散化)
14     :param x: 待舍入值 (整数)
15     :param q: 输入模数 (整数)
16     :param rq: 输出模数 (整数)
17     """
18
19     z = int(floor(1.* rq * x / q + rq/(2*q))) % rq) # 将 'round' 改为 '
20     floor', 使 0.5 向上舍入

```

```

21     if (z >= rq/2):
22         z -= rq
23
24     return z
25
26 def mod_centered(x, q):
27     """ 模 q 的中心化约减 (即表示在  $-q/2 \dots q/2$  中)
28     :param x: 待舍入值 (整数)
29     :param q: 输入模数 (整数)
30     """
31     a = x % q
32     if a < q/2:
33         return a
34     return a - q
35
36 def build_mod_switching_error_law(q, rq):
37     """ 构造误差分布: 模 q 均匀值经模数切换再切回时引入的差值分布
38     :param q: 原始模数 (整数)
39     :param rq: 中间模数 (整数)
40     """
41     D = {}
42     V = {}
43     for x in range(-int(q/2), int(q/2)):
44         y = mod_switch(x, q, rq)
45         z = mod_switch(y, rq, q)
46         d = mod_centered(x - z, q)
47         D[d] = D.get(d, 0) + 1./q
48         V[y] = V.get(y, 0) + 1
49
50     return D
51
52 def law_convolution(A, B):
53     """ 构造两个分布的卷积 (两个输入分布中独立变量之和)
54     :param A: 第一个输入分布 (字典)
55     :param B: 第二个输入分布 (字典)
56     """
57
58     C = {}
59     for a in A:
60         for b in B:
61             c = a+b
62             C[c] = C.get(c, 0) + A[a] * B[b]
63     return C
64
65 def law_convolution_neg(A, B):

```

```

66     """ 构造两个分布的差卷积 (两个输入分布中独立变量之差)
67     :param A: 第一个输入分布 (字典)
68     :param B: 第二个输入分布 (字典)
69     """
70
71     C = {}
72     for a in A:
73         for b in B:
74             c = a-b
75             C[c] = C.get(c, 0) + A[a] * B[b]
76     return C
77
78 def law_product(A, B):
79     """ 构造两个输入分布中独立变量乘积的分布
80     :param A: 第一个输入分布 (字典)
81     :param B: 第二个输入分布 (字典)
82     """
83     C = {}
84     for a in A:
85         for b in B:
86             c = a*b
87             C[c] = C.get(c, 0) + A[a] * B[b]
88     return C
89
90 dist_rate = -100000 # 加入此值以避免过早剪枝
91 def clean_dist(A):
92     """ 清理分布以加速后续计算 (丢弃支撑集中概率小于  $2^{\text{dist\_rate}}$  的元
93         素)
94     :param A: 输入分布 (字典)
95     """
96     B = {}
97     for (x, y) in A.items():
98         if y>2**(dist_rate):
99             B[x] = y
100     return B
101
102 def iter_law_convolution(A, i):
103     """ 计算分布的 i 重卷积 (使用倍加法)
104     :param A: 输入分布 (字典)
105     :param i: 重数 (整数)
106     """
107     D = {0: 1.0}
108     i_bin = bin(i)[2:] # i 的二进制表示
109     for ch in i_bin:
110         D = law_convolution(D, D)

```

```

110         D = clean_dist(D)
111         if ch == '1':
112             D = law_convolution(D, A)
113             D = clean_dist(D)
114     return D
115
116 def convolution_remove_dependency(A, B, q, p):
117     normalizer={}
118     maxa=int(q/p)
119     for a in A:
120         normalizer[a%maxa]=normalizer.get(a%maxa,0)+A[a]
121
122     C={}
123     for a in A:
124         for b in B:
125             c=a-b
126             if (c%maxa==0):
127                 C[c] = C.get(c, 0) + A[a] * B[b]/normalizer[a%maxa]
128     return C
129
130 def tail_probability(D, t):
131     '''
132     从 D 中抽取的值之绝对值严格大于 t 的概率
133     :param D: 分布 (字典)
134     :param t: 尾部参数 (整数)
135     '''
136     s = 0
137     ma = max(D.keys())
138     if t >= ma:
139         return 0
140     for i in reversed(range(int(ceil(t)), ma)): # 反向求和以提高数值精
141         s += D.get(i, 0) + D.get(-i, 0)
142     return s
143
144 def build_uniform_law(k):
145     """ 以字典形式构造均匀分布
146     :param k: 整数
147     :param x: 整数
148     :returns: 字典 {x:p_k(x) for x in {-k..k}}
149     """
150     D = {}
151     for i in range(-k, k):
152         D[i] = 1/(2*k)
153     return D

```

```

154
155 def get_failure_prob(D_s, q, p, T, k, ell, n):
156
157     # 计算舍入误差分布
158     D_e = build_mod_switching_error_law(q, p)
159
160     assert D_e == D_s # 对 Mithril 参数集成立
161
162     # 计算共享秘密误差分布
163     se = law_product(D_s, D_e) # 输入多项式系数单项乘积的分布
164
165     prob_secret_success_polys = ell*[0]
166     for i in range(ell):
167
168         # 对第 i 行, 有 k-i 个不含因子 (y+2) 的多项式, 以及 i 个含 (y
169             +2) 并额外获得因子 3 的多项式
170         # 总计 (k-i) + 3*i = k+2*i
171         se2 = iter_law_convolution(se, (k+2*i)*n) # s'*e 的累积多项式系
172             数分布, 假设各系数相互独立
173         se2 = law_convolution_neg(se2, se2) # s'*e - e'*s 的累积多项式
174             系数分布
175         e2 = build_mod_switching_error_law(q, T) # 密文误差分布
176         D = law_convolution(se2, e2) # 共享秘密系数的分布
177
178         prob_coeff_fail = tail_probability(D, q/4.) # 单个共享秘密比特
179             的失败概率
180         a = decimal.Decimal(prob_coeff_fail)
181         prob_secret_success = (1-a)**n # 该多项式所有比特均成功的概率
182         prob_secret_success_polys[i] = prob_secret_success
183
184     # 计算所有私钥多项式均成功的概率
185     prob_success = decimal.Decimal(1)
186     for i in range(ell):
187         prob_success *= prob_secret_success_polys[i]
188
189     prob_fail = log(1-prob_success, 2)
190
191     return prob_fail
192
193 def main():
194     D_s = build_uniform_law(2)
195     prob128 = get_failure_prob(D_s, 2**13, 2**11, 2**3, 5, 1, 128)
196     print('128 比特安全等级的失败概率: ', prob128)
197     prob256 = get_failure_prob(D_s, 2**13, 2**11, 2**3, 9, 2, 128)
198     print('256 比特安全等级的失败概率: ', prob256)

```

```

195     prob512 = get_failure_prob(D_s, 2**13, 2**11, 2**8, 17, 4, 128)
196     print('512 比特安全等级的失败概率: ', prob512)
197
198     main()

```

B 检查清单

以下检查清单依据提交要求文档编制，以便定位相关内容。

• 1. 算法描述

- 完整的算法运行流程：第 4 节，尤其是图 1。
- PKE 算法 KeyGen、Encrypt 和 Decrypt：第 4 节，图 1。
- KEM 算法 KeyGen、Encaps 和 Decaps：第 4 节，图 1。
- 数学记号与基本定义：第 3.1 节。
- LWE/LWR 背景与公式：第 3.2 节。
- 根式环结构与公式：第 3.3、3.4 节。
- 根式环上的结构化乘法：第 3.4 节，尤其是式 (6)。
- 所有安全强度等级的算法实例参数：第 4 节，表 2。
- Mithril-128、Mithril-256 和 Mithril-512 的参数：第 4 节，表 2。
- 密钥、密文和共享秘密的长度：第 4 节，表 2。
- 参数表中的失败概率与安全性估计：第 4 节，表 2。
- 采样、随机性、种子、哈希/XOF 的使用以及输入/输出处理：第 3.1、4 节。
- 支撑算法描述的表格：第 2.2 节的表 1；第 4 节的表 2；第 5 节的表 3；以及第 6 节的表 4-8。
- 算法图示：第 4 节，图 1。

• 2. 设计原理

- 主要设计思想与依据：第 1、2.1 节。
- 数学困难问题：第 2.1、3.4、5.1.3 节。
- 作为底层假设的 RR-LWR：第 1、2.1、3.4、5.1.3 节。
- 参数选择准则：第 1、2.1 节及表 2。
- 根式环结构的选择：第 1、2.1、3.4 节。
- n 、 k 、 p 、 q 、 t 和 ℓ 的选择：第 2.1 节及表 2。
- 2 的幂模数与均匀采样的使用：第 1、2.1 节。
- 失败概率及其安全性影响的详细分析：第 2.1、5.2.1 节及附录 A.2。

- 其他设计考虑：第 2.1 节，包括采样、部分环元素、Fujisaki–Okamoto 变换和高保障实现。
- 与现有工作的关系及比较依据：第 2.2 节。
- 3. 安全性声明与分析
 - 算法安全性声明：第 5 节。
 - 理论安全性：第 5.1 节。
 - RR-LWR 困难问题：第 5.1.3 节。
 - Mithril.PKE 的 IND-CPA 安全性：第 5.1.1 节。
 - ROM 中 Mithril.KEM 的 IND-CCA 安全性：第 5.1.2 节。
 - QROM 中 Mithril.KEM 的 IND-CCA 安全性：第 5.1 节。
 - 安全模型与安全性证明：第 5.1、5.1.1、5.1.2 节。
 - 量子模型安全性分析：第 5.1 节。
 - 实践安全性方法：第 5.1.4 节。
 - 各实例已知经典与量子攻击的复杂度：第 5.1.4 节的表 3；第 4 节的表 2；以及附录 A.1。
 - 可复现的安全性估计脚本：附录 A.1。
 - 解密失败攻击：第 5.2.1 节。
 - 侧信道攻击及常数时间/掩码相关考虑：第 5.2.2 节。
- 4. 性能评估
 - 性能分析：第 6.1、6.2 节。
 - 算术实现分析：第 6.1 节。
 - 参考实现的算术运算：第 6.1 节。
 - AVX2 实现的算术运算：第 6.1 节。
 - 采样与扩展开销分析：第 6.2 节。
 - 根式环乘积开销分析：第 6.2 节。
 - 操作级性能分析：第 6.2 节。
 - 通信与存储开销分析：第 6.2 节及表 2。
 - 所有算法实例在主流 64 位 PC 处理器上的性能测试结果：第 6.3 节。
 - 实现平台的详细配置：第 6.3 节“实验设置”。文档给出了 CPU、时钟频率、指令集、操作系统、编译器、实现语言和基准测试方法。
 - PKE 性能结果：第 6.3 节，表 4。
 - KEM 性能结果：第 6.3 节，表 5。

- 算术运算细分结果：第 6.3 节，表 6、7。
 - 与 Kyber AVX2 实现的比较：第 6.3 节，表 8。
- 5. 特性声明
 - 特性声明，包括创新性、简洁性、灵活性、兼容性、可扩展性及不同平台上的性能优势：第 1 节。

C 专业术语对照表

下表列出本文档使用的主要专业术语，按英文术语的字母顺序排列。

英文术语	中文译名
adaptive chosen-ciphertext attack	适应性选择密文攻击
algebraic degree	代数次数
algorithm agility	算法敏捷性
authenticated key establishment	认证密钥建立
canonical embedding	典范嵌入
centered binomial distribution	中心二项分布
centered representative	中心化代表元
Chinese Remainder Theorem (CRT)	中国剩余定理 (CRT)
chosen-ciphertext security	选择密文安全性
ciphertext	密文
ciphertext compression	密文压缩
coefficient-wise operation	逐系数运算
collision resistance	抗碰撞性
Core-SVP	Core-SVP
cyclotomic field	分圆域
cyclotomic polynomial	分圆多项式
cyclotomic ring	分圆环
decapsulation	解封装
decision Learning With Errors	判定型误差学习
decision Learning With Rounding	判定型舍入学习
decryption failure	解密失败
decryption-failure attack	解密失败攻击
deterministic encryption	确定性加密
digital signature	数字签名
domain separation	域分离
encapsulation	封装

接下页

英文术语	中文译名
ephemeral key establishment	临时密钥建立
extendable-output function (XOF)	可扩展输出函数 (XOF)
failure probability	失败概率
fault attack	故障攻击
Fujisaki–Okamoto transformation	Fujisaki–Okamoto 变换
high-assurance implementation	高保障实现
hybrid mode	混合模式
ideal lattice	理想格
implicit rejection	隐式拒绝
indistinguishability under adaptive chosen-ciphertext attack (IND-CCA)	适应性选择密文攻击下的不可区分性 (IND-CCA)
indistinguishability under chosen-plaintext attack (IND-CPA)	选择明文攻击下的不可区分性 (IND-CPA)
KEM–DEM encryption	KEM–DEM 加密
key confirmation	密钥确认
key encapsulation mechanism (KEM)	密钥封装机制 (KEM)
lattice reduction	格约化
Learning With Errors (LWE)	误差学习 (LWE)
Learning With Rounding (LWR)	舍入学习 (LWR)
masking	掩码防护
Module Learning With Errors (Module-LWE)	模上误差学习 (Module-LWE)
Module Learning With Rounding (Module-LWR)	模上舍入学习 (Module-LWR)
module rank	模秩
monic polynomial	首一多项式
negacyclic convolution	负循环卷积
Number Theoretic Transform (NTT)	数论变换 (NTT)
plaintext checking	明文检查
post-quantum cryptography	后量子密码学
power-of-two modulus	2 的幂模数
primitive root of unity	本原单位根
pseudorandom generator (PRG)	伪随机生成器 (PRG)
public-key encryption (PKE)	公钥加密 (PKE)
quantum random oracle model (QROM)	量子随机预言机模型 (QROM)
quotient ring	商环
radical extension	根式扩张

接下页

英文术语	中文译名
radical ring	根式环
Radical Ring Learning With Rounding (RR-LWR)	根式环舍入学习 (RR-LWR)
random oracle model (ROM)	随机预言机模型 (ROM)
reconciliation	协调
rejection sampling	拒绝采样
Ring Learning With Errors (Ring-LWE)	环上误差学习 (Ring-LWE)
Ring Learning With Rounding (Ring-LWR)	环上舍入学习 (Ring-LWR)
rounding error	舍入误差
search Learning With Errors	搜索型误差学习
search Learning With Rounding	搜索型舍入学习
secret distribution	秘密分布
session key	会话密钥
shared secret	共享秘密
side-channel attack	侧信道攻击
signed representative	有符号代表元
statistical distance	统计距离
uniform distribution	均匀分布

参考文献

- [ABC⁺23] Aikata, Andrea Basso, Gaëtan Cassiers, Ahmet Can Mert, and Sujoy Sinha Roy. Kavach: Lightweight masking techniques for polynomial arithmetic in lattice-based cryptography. *IACR TCHES*, 2023(3):366–390, 2023. [doi:10.46586/tches.v2023.i3.366-390](https://doi.org/10.46586/tches.v2023.i3.366-390).
- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – a new hope. In *Proceedings of the 25th USENIX Security Symposium*, pages 327–343. USENIX Association, 2016.
- [AKPW13] Joël Alwen, Stephan Krenn, Krzysztof Pietrzak, and Daniel Wichs. Learning with rounding, revisited - new reduction, properties and applications. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 57–74. Springer, Berlin, Heidelberg, August 2013. [doi:10.1007/978-3-642-40041-4_4](https://doi.org/10.1007/978-3-642-40041-4_4).
- [AKSvV22] Melissa Azouaoui, Yulia Kuzovkova, Tobias Schneider, and Christine van Vredendaal. Post-quantum authenticated encryption against chosen-ciphertext side-

- channel attacks. *IACR TCHES*, 2022(4):372–396, 2022. doi:[10.46586/tches.v2022.i4.372-396](https://doi.org/10.46586/tches.v2022.i4.372-396).
- [APS15] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015. doi:[doi:10.1515/jmc-2015-0016](https://doi.org/10.1515/jmc-2015-0016).
- [APUPG17] Troncoso-Pastoriza Alberto Pedrouzo-Ulloa, Juan Ramón and Fernando Pérez-González. Multivariate cryptosystems for secure processing of multidimensional signals. *ArXiv*, abs/1712.00848, 2017.
- [BBD⁺26] Andrea Basso, Joppe W. Bos, Jan-Pieter D’Anvers, Angshuman Karmakar, Jose Maria Bermudo Mera, Joost Renes, Sujoy Sinha Roy, Frederik Vercauteren, Peng Wang, Yuewu Wang, Shicong Zhang, and Chenxin Zhong. Using learning with rounding to instantiate post-quantum cryptographic algorithms. *ACM Trans. Embed. Comput. Syst.*, 2026. URL: <https://eprint.iacr.org/2025/1382>, doi:[10.1145/3807514](https://doi.org/10.1145/3807514).
- [BBE⁺18] Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the GLP lattice-based signature scheme at any order. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 354–384. Springer, Cham, April / May 2018. doi:[10.1007/978-3-319-78375-8_12](https://doi.org/10.1007/978-3-319-78375-8_12).
- [BCV20] Carl Bootland, Wouter Castryck, and Frederik Vercauteren. On the security of the multivariate ring learning with errors problem. In *ANTS-XIV*, volume 4, pages 57–71, 2020. doi:[10.2140/obs.2020.4.57](https://doi.org/10.2140/obs.2020.4.57).
- [BDK⁺21] Michiel Van Beirendonck, Jan-Pieter D’ anvers, Angshuman Karmakar, Josep Balasch, and Ingrid Verbauwhede. A side-channel-resistant implementation of SABER. *J. Emerg. Technol. Comput. Syst.*, 17(2), 2021. doi:[10.1145/3429983](https://doi.org/10.1145/3429983).
- [BDPR98] Mihir Bellare, Anand Desai, David Pointcheval, and Phillip Rogaway. Relations among notions of security for public-key encryption schemes. In Hugo Krawczyk, editor, *CRYPTO’98*, volume 1462 of *LNCS*, pages 26–45. Springer, Berlin, Heidelberg, August 1998. doi:[10.1007/BFb0055718](https://doi.org/10.1007/BFb0055718).
- [BGM⁺16] Andrej Bogdanov, Siyao Guo, Daniel Masny, Silas Richelson, and Alon Rosen. On the hardness of learning with rounding over small modulus. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part I*, volume 9562 of *LNCS*, pages 209–224. Springer, Berlin, Heidelberg, January 2016. doi:[10.1007/978-3-662-49096-9_9](https://doi.org/10.1007/978-3-662-49096-9_9).

- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012. doi:[10.1145/2090236.2090262](https://doi.org/10.1145/2090236.2090262).
- [BHH⁺19] Nina Bindel, Mike Hamburg, Kathrin Hövelmanns, Andreas Hülsing, and Edoardo Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *TCC 2019, Part II*, volume 11892 of *LNCS*, pages 61–90. Springer, Cham, December 2019. doi:[10.1007/978-3-030-36033-7_3](https://doi.org/10.1007/978-3-030-36033-7_3).
- [BN00] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In Tatsuo Okamoto, editor, *ASIACRYPT 2000*, volume 1976 of *LNCS*, pages 531–545. Springer, Berlin, Heidelberg, December 2000. doi:[10.1007/3-540-44448-3_41](https://doi.org/10.1007/3-540-44448-3_41).
- [BPR12] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 719–737. Springer, Berlin, Heidelberg, April 2012. doi:[10.1007/978-3-642-29011-4_42](https://doi.org/10.1007/978-3-642-29011-4_42).
- [BRvV22] Joppe W. Bos, Joost Renes, and Christine van Vredendaal. Post-quantum cryptography with contemporary co-processors: Beyond Kronecker, Schönhage-Strassen & Nussbaumer. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3683–3697, Boston, MA, 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/bos>.
- [BRVW26] Joppe W. Bos, Joost Renes, Frederik Vercauteren, and Peng Wang. Structured module lattice-based cryptography: Fine-grained parameter selection without compromising performance. *IACR Transactions on Cryptographic Hardware and Embedded Systems (to appear)*, 2026(3), 2026. URL: <https://eprint.iacr.org/2026/098>.
- [CCHY24] Jung Hee Cheon, Hyeongmin Choe, Dongyeon Hong, and MinJune Yi. SMAUG: Pushing lattice-based key encapsulation mechanisms to the limits. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023*, volume 14201 of *LNCS*, pages 127–146. Springer, Cham, August 2024. doi:[10.1007/978-3-031-53368-6_7](https://doi.org/10.1007/978-3-031-53368-6_7).
- [CGV14] Jean-Sébastien Coron, Johann Großschädl, and Praveen Kumar Vadnala. Secure conversion between Boolean and arithmetic masking of any order. In Lejla Batina and Matthew Robshaw, editors, *CHES 2014*, volume 8731 of *LNCS*, pages 188–205. Springer, Berlin, Heidelberg, September 2014. doi:[10.1007/978-3-662-44709-3_11](https://doi.org/10.1007/978-3-662-44709-3_11).

- [CHK⁺21] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih, and Bo-Yin Yang. NTT multiplication for NTT-unfriendly rings. *IACR TCHES*, 2021(2):159–188, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/8791>, doi: [10.46586/tches.v2021.i2.159-188](https://doi.org/10.46586/tches.v2021.i2.159-188).
- [CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 398–412. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-48405-1_26](https://doi.org/10.1007/3-540-48405-1_26).
- [CN11] Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, December 2011. doi:[10.1007/978-3-642-25385-0_1](https://doi.org/10.1007/978-3-642-25385-0_1).
- [Coo66] S. Cook. *On the minimum computation time of functions*. PhD thesis, Harvard University, 1966.
- [CT65] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of computation*, 19(90):297–301, 1965.
- [DB22] Jan-Pieter D’Anvers and Senne Batsleer. Multitarget decryption failure attacks and their application to Saber and Kyber. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part I*, volume 13177 of *LNCS*, pages 3–33. Springer, Cham, March 2022. doi:[10.1007/978-3-030-97121-2_1](https://doi.org/10.1007/978-3-030-97121-2_1).
- [DFMS19] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Security of the Fiat-Shamir transformation in the quantum random-oracle model. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 356–383. Springer, Cham, August 2019. doi: [10.1007/978-3-030-26951-7_13](https://doi.org/10.1007/978-3-030-26951-7_13).
- [DFMS21] Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. Cryptology ePrint Archive, Report 2021/280, 2021. URL: <https://eprint.iacr.org/2021/280>.
- [DGJ⁺19] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on IND-CCA secure lattice-based schemes. In Dongdai Lin and Kazue Sako, editors, *PKC 2019, Part II*, volume 11443 of *LNCS*, pages 565–598. Springer, Cham, April 2019. doi:[10.1007/978-3-030-17259-6_19](https://doi.org/10.1007/978-3-030-17259-6_19).
- [DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A

- lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018. URL: <https://tches.iacr.org/index.php/TCHES/article/view/839>, doi: [10.13154/tches.v2018.i1.238-268](https://doi.org/10.13154/tches.v2018.i1.238-268).
- [DKR⁺20] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Frederik Vercauteren, Jose Maria Bermudo Mera, Michiel Van Beirendonck, and Andrea Basso. SABER. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [DKRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18*, volume 10831 of *LNCS*, pages 282–305. Springer, Cham, May 2018. doi:[10.1007/978-3-319-89339-6_16](https://doi.org/10.1007/978-3-319-89339-6_16).
- [DRV20] Jan-Pieter D’Anvers, Mélissa Rossi, and Fernando Virdia. (One) failure is not an option: Bootstrapping the search for failures in lattice-based encryption schemes. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 3–33. Springer, Cham, May 2020. doi:[10.1007/978-3-030-45727-3_1](https://doi.org/10.1007/978-3-030-45727-3_1).
- [DVV28] Jan-Pieter D’Anvers, Ingrid Verbauwhede, and Frédérik Vercauteren. Design and security analysis of lattice-based post-quantum encryption, 2021-05-28.
- [FO99] Eiichi Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999. doi:[10.1007/3-540-48405-1_34](https://doi.org/10.1007/3-540-48405-1_34).
- [FVR⁺22] Tim Fritzmann, Michiel Van Beirendonck, Debapriya Basu Roy, Patrick Karl, Thomas Schamberger, Ingrid Verbauwhede, and Georg Sigl. Masked accelerators and instruction set extensions for post-quantum cryptography. *IACR TCHES*, 2022(1):414–460, 2022. doi:[10.46586/tches.v2022.i1.414-460](https://doi.org/10.46586/tches.v2022.i1.414-460).
- [HCCSS24] Jung Hee Cheon, Hyeongmin Choe, Jungjoo Seo, and Hyeon Seong. SMAUG(-T), revisited: Timing-secure, more compact, less failure. *IEEE Access*, 12:188386–188397, 2024. doi:[10.1109/ACCESS.2024.3511346](https://doi.org/10.1109/ACCESS.2024.3511346).
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017. doi:[10.1007/978-3-319-70500-2_12](https://doi.org/10.1007/978-3-319-70500-2_12).

- [KDvB⁺22] Suparna Kundu, Jan-Pieter D’Anvers, Michiel van Beirendonck, Angshuman Karmakar, and Ingrid Verbauwhede. Higher-order masked Saber. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 93–116. Springer, Cham, September 2022. [doi:10.1007/978-3-031-14791-3_5](https://doi.org/10.1007/978-3-031-14791-3_5).
- [KKV23] Suparna Kundu, Angshuman Karmakar, and Ingrid Verbauwhede. On the masking-friendly designs for post-quantum cryptography. In Francesco Regazzoni, Bodhisatwa Mazumdar, and Sri Parameswaran, editors, *Security, Privacy, and Applied Cryptography Engineering - 13th International Conference, SPACE 2023, Roorkee, India, December 14-17, 2023, Proceedings*, volume 14412 of *Lecture Notes in Computer Science*, pages 162–184. Springer, 2023. [doi:10.1007/978-3-031-51583-5_10](https://doi.org/10.1007/978-3-031-51583-5_10).
- [KNK⁺25] Suparna Kundu, Quinten Norga, Angshuman Karmakar, Shreya Gangopadhyay, Jose Maria Bermudo Mera, and Ingrid Verbauwhede. Scabbard: An exploratory study on hardware aware design choices of learning with rounding-based key encapsulation mechanisms. *ACM Trans. Embed. Comput. Syst.*, 24(1):10:1–10:40, 2025. [doi:10.1145/3696208](https://doi.org/10.1145/3696208).
- [KO62] Anatolii Alekseevich Karatsuba and Yu P Ofman. Multiplication of many-digital numbers by automatic computers. In *Doklady Akademii Nauk*, volume 145, pages 293–294. Russian Academy of Sciences, 1962.
- [Kro82] L. Kronecker. Grundzüge einer arithmetischen Theorie der algebraischen Grössen. *Journal für die reine und angewandte Mathematik*, 92:1–122, 1882.
- [KSS⁺20] Veronika Kuchta, Amin Sakzad, Damien Stehlé, Ron Steinfeld, and Shifeng Sun. Measure-rewind-measure: Tighter quantum random oracle model proofs for one-way to hiding and CCA security. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 703–728. Springer, Cham, May 2020. [doi:10.1007/978-3-030-45727-3_24](https://doi.org/10.1007/978-3-030-45727-3_24).
- [Laa15] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 3–22. Springer, Berlin, Heidelberg, August 2015. [doi:10.1007/978-3-662-47989-6_1](https://doi.org/10.1007/978-3-662-47989-6_1).
- [LMvdP15] Thijs Laarhoven, Michele Mosca, and Joop van de Pol. Finding shortest lattice vectors faster using quantum search. *DCC*, 77(2-3):375–400, 2015. [doi:10.1007/s10623-015-0067-5](https://doi.org/10.1007/s10623-015-0067-5).
- [LPR10] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*,

volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010. doi:[10.1007/978-3-642-13190-5_1](https://doi.org/10.1007/978-3-642-13190-5_1).

- [LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *DCC*, 75(3):565–599, 2015. doi:[10.1007/s10623-014-9938-4](https://doi.org/10.1007/s10623-014-9938-4).
- [Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009. doi:[10.1007/978-3-642-10366-7_35](https://doi.org/10.1007/978-3-642-10366-7_35).
- [Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012. doi:[10.1007/978-3-642-29011-4_43](https://doi.org/10.1007/978-3-642-29011-4_43).
- [MKKV21] Jose Maria Bermudo Mera, Angshuman Karmakar, Suparna Kundu, and Ingrid Verbauwhede. Scabbard: a suite of efficient learning with rounding key-encapsulation mechanisms. *IACR TCHES*, 2021(4):474–509, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/9073>, doi:[10.46586/tches.v2021.i4.474-509](https://doi.org/10.46586/tches.v2021.i4.474-509).
- [MTK⁺20] Jose Maria Bermudo Mera, Furkan Turan, Angshuman Karmakar, Sujoy Sinha Roy, and Ingrid Verbauwhede. Compact domain-specific co-processor for accelerating module lattice-based KEM. In *57th ACM/IEEE Design Automation Conference, DAC 2020, San Francisco, CA, USA, July 20-24, 2020*, pages 1–6. IEEE, 2020. doi:[10.1109/DAC18072.2020.9218727](https://doi.org/10.1109/DAC18072.2020.9218727).
- [Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, 2015. doi:[10.6028/NIST.FIPS.202](https://doi.org/10.6028/NIST.FIPS.202).
- [NR23] Parker Newton and Silas Richelson. A lower bound for proving hardness of learning with rounding with polynomial modulus. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 805–835. Springer, Cham, August 2023. doi:[10.1007/978-3-031-38554-4_26](https://doi.org/10.1007/978-3-031-38554-4_26).
- [OTF⁺20] Hiroki Okada, Atsushi Takayasu, Kazuhide Fukushima, Shinsaku Kiyomoto, and Tsuyoshi Takagi. A compact digital signature scheme based on the module-LWR problem. In Weizhi Meng, Dieter Gollmann, Christian Damsgaard Jensen, and Jianying Zhou, editors, *ICICS 20*, volume 11999 of *LNCS*, pages 73–90. Springer, Cham, August 2020. doi:[10.1007/978-3-030-61078-4_5](https://doi.org/10.1007/978-3-030-61078-4_5).
- [Pei16] Chris Peikert. A decade of lattice cryptography. *Foundations and Trends® in Theoretical Computer Science*, 10(4):283–424, 2016. doi:[10.1561/04000000074](https://doi.org/10.1561/04000000074).

- [PJP⁺22] Seunghwan Park, Chi-Gon Jung, Aesun Park, Joongeun Choi, and Honggoo Kang. TiGER: Tiny bandwidth key encapsulation mechanism for easy miGration based on RLWE(R). Cryptology ePrint Archive, Report 2022/1651, 2022. URL: <https://eprint.iacr.org/2022/1651>.
- [PN25] Rafaël Del Pino and Guilhem Niot. Finally! A compact lattice-based threshold signature. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 169–199. Springer, Cham, May 2025. doi: [10.1007/978-3-031-91826-1_6](https://doi.org/10.1007/978-3-031-91826-1_6).
- [Pol71] John M. Pollard. The fast Fourier transform in a finite field. *Mathematics of Computation*, 25(114):365–374, 1971.
- [PR13] Emmanuel Prouff and Matthieu Rivain. Masking against side-channel attacks: A formal security proof. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 142–159. Springer, Berlin, Heidelberg, May 2013. doi: [10.1007/978-3-642-38348-9_9](https://doi.org/10.1007/978-3-642-38348-9_9).
- [PUTPG⁺21] Alberto Pedrouzo-Ulloa, Juan Ramón Troncoso-Pastoriza, Nicolas Gama, Mariya Georgieva, and Fernando Pérez-González. Revisiting multivariate ring learning with errors and its applications on lattice-based cryptography. *Mathematics*, 9(8), 2021. URL: <https://www.mdpi.com/2227-7390/9/8/858>, doi: [10.3390/math9080858](https://doi.org/10.3390/math9080858).
- [PUTPPG15] Alberto Pedrouzo-Ulloa, Juan Ramón Troncoso-Pastoriza, and Fernando Pérez-González. Multivariate lattices for encrypted image processing. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1707–1711, 2015. doi: [10.1109/ICASSP.2015.7178262](https://doi.org/10.1109/ICASSP.2015.7178262).
- [PUTPPG16] Alberto Pedrouzo-Ulloa, Troncoso-Pastoriza, and Fernando Pérez-González. On ring learning with errors over the tensor product of number fields, 2016. URL: <https://arxiv.org/abs/1607.05244>.
- [RCDB24] Prasanna Ravi, Anupam Chattopadhyay, Jan Pieter D’Anvers, and Anubhab Baksi. Side-channel and fault-injection attacks over lattice-based post-quantum schemes (Kyber, Dilithium): Survey and new results. *ACM Trans. Embed. Comput. Syst.*, 23(2), March 2024. doi: [10.1145/3603170](https://doi.org/10.1145/3603170).
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005. doi: [10.1145/1060590.1060603](https://doi.org/10.1145/1060590.1060603).
- [Reg10] Oded Regev. The learning with errors problem (invited survey). In *IEEE Conference on Computational Complexity*, pages 191–204, 2010. doi: [10.1109/CCC.2010.26](https://doi.org/10.1109/CCC.2010.26).

- [RRCB20] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based PKE and kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(3):307–335, 2020. URL: <https://doi.org/10.13154/tches.v2020.i3.307-335>, doi:10.13154/TCHES.V2020.I3.307-335.
- [SE94] Claus-Peter Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Math. Program.*, 66:181–199, 1994. doi:10.1007/BF01581144.
- [Too63] Andrei L Toom. The complexity of a scheme of functional elements realizing the multiplication of integers, published in soviet math (translations of dokl. adad. nauk. sssr), 4, 1963.
- [XPR⁺22] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, David Oswald, Wang Yao, and Zhiming Zheng. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of Kyber. *IEEE Transactions on Computers*, 71(9):2163–2176, 2022. doi:10.1109/TC.2021.3122997.